

Master_CheatSheet_OpenNotes

Master Open-Notes Cheat Sheet — ML End-Sem

Dense, scannable reference for the open-notes exam. Every key formula, every decision rule. Pre-mid-sem (topics 1–8) and post-mid-sem (topics 9–13).

1 • Probability

- $P(A | B) = P(A \cap B) / P(B)$. **Bayes:** $P(A | B) = P(B | A)P(A) / P(B)$. **Total prob:** $P(B) = \sum_i P(B | A_i)P(A_i)$.
- **Independent:** $P(A \cap B) = P(A)P(B) \Leftrightarrow P(A | B) = P(A)$. **Conditionally indep** $A \perp B | C$: $P(A \cap B | C) = P(A | C)P(B | C)$.
- **Linearity** (always): $E[aX + bY + c] = aE[X] + bE[Y] + c$. **Var addition** (only if indep): $\text{Var}(X + Y) = \text{Var}(X) + \text{Var}(Y)$. $\text{Var}(aX) = a^2 \text{Var}(X)$.
- $\text{Var}(X) = E[X^2] - (E[X])^2$. $\text{Cov}(X, Y) = E[(X - EX)(Y - EY)]$.
- Bernoulli(p): $E = p$, $V = p(1 - p)$. Bin(n, p): $E = np$, $V = np(1 - p)$. Geo(p): $E = 1/p$. Unif $\{1..n\}$: $E = (n + 1)/2$. Fair die: $E = 3.5$, $V = 35/12$.

2 • Linear Algebra

- $\langle u, v \rangle = u^\top v$. $\|v\| = \sqrt{v^\top v}$. $\cos \theta = u^\top v / (\|u\| \|v\|)$.
- $\{v_i\}$ indep $\Leftrightarrow \sum c_i v_i = 0 \Rightarrow c_i = 0$. $\mathbb{R}^n$: max n indep vectors.
- **Projection onto col(X):** $\hat{y} = X(X^\top X)^{-1}X^\top y = Hy$. $H^2 = H$, $H^\top = H$.
- $(AB)^\top = B^\top A^\top$, $(AB)^{-1} = B^{-1}A^{-1}$. \$ + = \$ cols.
- Eigen: $Av = \lambda v$. Solve $\det(A - \lambda I) = 0$. 2×2 : $\lambda^2 - (\text{tr})\lambda + \det = 0$.
- **PSD** $A \succeq 0 \Leftrightarrow x^\top Ax \geq 0 \forall x \Leftrightarrow \text{all } \lambda \geq 0$. Symmetric $\Rightarrow$ real λ 's, orthogonal eigenvectors, $A = U\Lambda U^\top$.

3 • Linear Regression (OLS)

- Model: $y = X\beta + \varepsilon$. Loss $L(\beta) = \frac{1}{2} \|X\beta - y\|^2$. Gradient: $X^\top (X\beta - y)$.
- **Normal eq:** $X^\top X\beta = X^\top y$. **Closed form:** $\hat{\beta} = (X^\top X)^{-1}X^\top y$.
- 1D: $\hat{\beta}_1 = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2}$, $\hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}$.
- $R^2 = 1 - \text{RSS}/\text{TSS}$; $\text{TSS} = \sum (y_i - \bar{y})^2$. $\hat{\sigma}^2 = \text{RSS}/(n - p)$.
- $t_j = \hat{\beta}_j / \text{SE}(\hat{\beta}_j)$. $|t_j| > 2 \approx p < 0.05$.
- **Gauss-Markov:** OLS is BLUE. Residual $\perp \text{col}(X)$ (geometric reading of normal eq).

4 • KNN, Validation, Bias-Variance

- KNN: vote/avg over k closest. Lazy, no params.

- **Bias-Var:** Test $MSE = \text{Bias}^2 + \text{Var} + \sigma^2$. Complexity $\uparrow \Rightarrow \text{Bias} \downarrow$ but $\text{Var} \uparrow$.
- Train low + test high + big gap $\Rightarrow$ **overfit** (var). Both high + small gap $\Rightarrow$ **underfit** (bias).
- **K-fold CV:** avg validation error over K folds. Tune hyperparams. Touch test only once.
- More data $\Rightarrow$ var $\downarrow$ but bias unchanged.

5 • Feature Engineering + Model Selection

- **Standardise** features before KNN / K-means / GD / regularised regression. $\tilde{x}_j = (x_j - \mu_j) / \sigma_j$.
- Polynomial features: $\phi(x) = (1, x, x^2, \dots)$ — linear in β still.
- One-hot: drop one column (avoid collinearity with intercept).
- $R_{\text{adj}}^2 = 1 - \frac{\text{RSS}/(n-p-1)}{\text{TSS}/(n-1)}$ — penalises useless features.
- **AIC** $= n \log(\text{RSS}/n) + 2p$ (lighter penalty, predictive).
- **BIC** $= n \log(\text{RSS}/n) + p \log n$ (stronger penalty, parsimony).
- Forward sel: empty $\rightarrow$ add best. Backward: full $\rightarrow$ drop worst.

6 • Convex Optimization

- Convex set: $\alpha x + (1 - \alpha)y \in C$ for $x, y \in C, \alpha \in [0, 1]$. Intersection of convex sets is convex.
- Convex function: $f(\alpha x + (1 - \alpha)y) \leq \alpha f(x) + (1 - \alpha)f(y)$. Chord $\geq$ graph.
- Char: $f(y) \geq f(x) + \langle \nabla f(x), y - x \rangle$ (1st-order). $\nabla^2 f \succeq 0$ (2nd-order).
- **Primal** (P): $\min f(x)$ s.t. $Ax \leq b, x \in \mathcal{X}$ convex.
- **Lagrangian**: $L(x, \alpha) = f(x) + \alpha^\top (Ax - b), \alpha \geq 0$.
- **Dual** (D): $\max_{\alpha \geq 0} F(\alpha)$ where $F(\alpha) = \inf_x L(x, \alpha)$. F concave; $d^* = p^*$ (strong duality).
- **KKT** (necessary & sufficient for convex): stationarity $\nabla_x L = 0$; primal feas $Ax^* \leq b$; dual feas $\alpha^* \geq 0$; **complementary slackness** $\alpha_i^* (a_i^\top x^* - b_i) = 0$.
- **GD update**: $\theta \leftarrow \theta - \eta \nabla f(\theta)$.

7 • SVM

- Labels $y \in \{-1, +1\}$. **Margin** $= 2/\|w\|$.
- **Hard-margin primal**: $\min \frac{1}{2} \|w\|^2$ s.t. $y^{(i)}(w^\top x^{(i)} + b) \geq 1$.
- **Soft-margin primal**: $\min \frac{1}{2} \|w\|^2 + C \sum_i \xi_i$ s.t. $y^{(i)}(w^\top x^{(i)} + b) \geq 1 - \xi_i, \xi_i \geq 0$.
- **Dual**: $\max_\alpha \sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y^{(i)} y^{(j)} K(x^{(i)}, x^{(j)})$ s.t. $\sum_i \alpha_i y^{(i)} = 0, 0 \leq \alpha_i \leq C$.
- $w^* = \sum_i \alpha_i^* y^{(i)} x^{(i)}$. **Support vector** $\Leftrightarrow \alpha_i^* > 0$.
- Soft margin SV: $0 < \alpha < C$ on boundary ($\xi = 0$); $\alpha = C$ inside/misclassified ($\xi > 0$).
- **Kernels**: linear $\langle x, z \rangle$; poly $(\langle x, z \rangle + c)^d$; RBF $e^{-\gamma \|x - z\|^2}$.
- Mercer: K valid $\Leftrightarrow$ Gram matrix PSD.

8 • Shrinkage — Ridge / Lasso

- **Ridge**: $\min \|y - X\beta\|^2 + \lambda \|\beta\|_2^2$. Closed: $\hat{\beta} = (X^\top X + \lambda I)^{-1} X^\top y$. Unique even when $p > n$. **No sparsity**.
- **Lasso**: $\min \|y - X\beta\|^2 + \lambda \|\beta\|_1$. No closed form. **Sparse** — feature selection by L1 corners.
- Both: shrink coefficients, biased but lower variance vs OLS.
- **Standardise** features first. **CV** for λ . **1-SE rule** for parsimony.

- Correlated features: Ridge averages, Lasso picks one.
- Elastic Net: $\lambda_1 \|\beta\|_1 + \lambda_2 \|\beta\|_2^2$ (best for correlated).

9 • Gradient Descent Variants + Logistic Regression

- $\theta \leftarrow \theta - \eta \nabla L$. Batch: all n , $O(nd)$ /step. SGD: 1 sample, $O(d)$. Mini-batch: B samples, $O(Bd)$.
- SGD unbiased: $E_i[\nabla L^{(i)}] = \nabla L$. Decaying $\eta_t = 1/\sqrt{t}$.
- **Logistic regression**: $h_\theta(x) = \sigma(\theta^\top x) = 1/(1 + e^{-\theta^\top x})$. Linear boundary at $\theta^\top x = 0$.
- Sigmoid: $g'(z) = g(z)(1 - g(z))$. $1 - g(z) = g(-z)$.
- **Cross-entropy** (convex): $J(\theta) = -\sum_i [y^{(i)} \log h + (1 - y^{(i)}) \log(1 - h)]$.
- Gradient: $\nabla_\theta J = X^\top (g(X\theta) - y)$. Same form as OLS.

10 • GDA + Naive Bayes (generative)

- **Bayes**: $\hat{y} = \arg \max_y P(x | y)P(y)$ (ignore $P(x)$).
- **GDA assumptions**: $y \sim \text{Ber}(\phi)$; $x | y = k \sim \mathcal{N}(\mu_k, \Sigma)$ with **shared** Σ .
- GDA MLE: $\hat{\phi}^* = \frac{\text{class } 1}{N}$; $\hat{\mu}_k^* = \frac{\text{class } k \text{ mean}}{N}$; $\hat{\Sigma}^* = (1/N) \sum_i (x^{(i)} - \mu_{y^{(i)}})(\dots)^\top$ (pooled).
- Shared $\Sigma \Rightarrow$ **linear** decision boundary $(\mu_1 - \mu_0)^\top \Sigma^{-1} x + c = 0$. Different $\Sigma \Rightarrow$ quadratic (QDA).
- **NB assumption**: $P(x | y) = \prod_j P(x_j | y)$ (cond indep given class). $(d + 1)K$ params.
- NB MLE (binary): $\phi_k = \text{count}(y = k)/N$; $\phi_{j|k} = \text{count}(x_j = 1, y = k)/\text{count}(y = k)$.
- **Laplace smoothing**: $\hat{\phi}_{j|k} = (c + 1)/(n_k + V)$, $V = \# \text{ values}$. Avoids zero-prob.

11 • Decision Trees + Ensembles

- Binary tree; internal nodes ask " $x_j \geq s$?"; leaves predict class/value.
- **Gini**: $G(S) = 1 - \sum_k p_k^2$. Binary: $G = 2p(1 - p)$. Pure $\Rightarrow 0$.
- **Weighted Gini** for a split: $\frac{|I^-|}{|I|} G(I^-) + \frac{|I^+|}{|I|} G(I^+) -$ minimise.
- Regression: replace Gini with **SSE** $= \sum_i (y^{(i)} - \bar{y}_I)^2$.
- Candidate splits: midpoints of sorted distinct values.
- Greedy recursive build. Stop when pure, $|I| < k$, or depth $> D$.
- **Post-prune**: $C_\alpha(T) = \text{train_err} + \alpha \cdot |\text{leaves}|$, α via CV.
- Trees are **high-variance**. Ensembling reduces var.
- **Bagging**: B bootstrap samples $\rightarrow$ train B models $\rightarrow$ vote/avg.
- **Random Forest** = Bagging + DT + per-split random subset of m features ($m = \sqrt{d}$ classif, $d/3$ regress).

12 • K-Means

- Objective: $\text{SSE}(C, \mu) = \sum_i \|x_i - \mu_{C(i)}\|^2$. Centroid: $\mu_c = \text{mean of cluster}$.
- **Algorithm** (Lloyd): repeat (a) assign $C(i) = \arg \min_\ell \|x_i - \mu_\ell\|^2$; (b) update centroid means.
- Monotone SSE decrease; converges to **local min** (multi-restart).
- Spread identity: $\sum_{i,j \in C_\ell} \|x_i - x_j\|^2 = 2m_\ell \sum_{i \in C_\ell} \|x_i - \mu_\ell\|^2$.
- **Pick K** : elbow in SSE-vs- K plot.

- Limitation: only **convex** clusters of similar size (Voronoi cells).

13 · PCA

- Centre data first. S = sample covariance.
- **Objective:** $\max_{\|u_1\|=1} u_1^\top S u_1$. **Solution:** u_1 = top eigenvector of S (eigenvalue λ_1).
- Top p components: $U_p = [u_1 | \dots | u_p]$. Project: $\tilde{x}_i = U_p^\top x_i$.
- **Explained variance ratio:** $\sum_{i=1}^p \lambda_i / \sum_{i=1}^d \lambda_i$. Common: 95%.
- Eigen recipe: $\det(A - \lambda I) = 0$, then $(A - \lambda I)v = 0$. $m \times m \Rightarrow \leq m$ roots.

Decision rules — “which method when?”

Situation	Use
Linear regression with $p > n$ or singular $X^\top X$	Ridge
Want to find which features matter (sparsity)	Lasso
Linear classifier, large n , robust	Logistic regression
Linear classifier, small n , Gaussian-like features	GDA
Text / high- d binary features	Naive Bayes
Need to express XOR / feature interactions	Decision tree
Off-the-shelf strong classifier with little tuning	Random Forest
Need maximum margin / non-linear boundary	Kernel SVM (RBF)
Want to cluster data	K-means (convex clusters)
Reduce dimension while preserving variance	PCA

Decision rules — “linear or quadratic boundary?”

- **Linear** in x : logistic regression, SVM (linear kernel), GDA (shared Σ), perceptron.
- **Quadratic:** QDA (different Σ_k), SVM with poly degree 2 kernel.
- **Non-linear arbitrary:** kernel SVM (RBF), decision trees, random forests.

Decision rules — “convex objective or not?”

- **Convex** (GD finds global min): OLS, ridge, lasso, logistic regression, SVM, K-means *per step* (not joint).
- **Non-convex:** K-means joint optimisation, neural nets, deep trees (greedy not optimal).

Common formulas to lift directly

- OLS: $\hat{\beta} = (X^\top X)^{-1} X^\top y$.
- Ridge: $\hat{\beta} = (X^\top X + \lambda I)^{-1} X^\top y$.

- $R^2 = 1 - \text{RSS}/\text{TSS}$. $\hat{\sigma}^2 = \text{RSS}/(n - p)$.
- Sigmoid: $\sigma(z) = 1/(1 + e^{-z})$. $\sigma'(z) = \sigma(z)(1 - \sigma(z))$.
- Cross-entropy gradient: $\nabla J = X^\top (\sigma(X\theta) - y)$.
- SGD update: $\theta \leftarrow \theta - \eta(\hat{y}^{(i)} - y^{(i)})x^{(i)}$.
- Gaussian $\mathcal{N}(\mu, \Sigma)$: $f(x) = (2\pi)^{-d/2} |\Sigma|^{-1/2} \exp(-\frac{1}{2}(x - \mu)^\top \Sigma^{-1}(x - \mu))$.
- SVM dual: $\max \sum \alpha_i - \frac{1}{2} \sum \alpha_i \alpha_j y^{(i)} y^{(j)} K(x^{(i)}, x^{(j)})$ s.t. $\sum \alpha_i y^{(i)} = 0$, $0 \leq \alpha_i \leq C$.
- $w^* = \sum_i \alpha_i^* y^{(i)} x^{(i)}$ (SVM).
- Gini: $1 - \sum_k p_k^2$. Binary: $2p(1 - p)$.
- Bagging error (toy, B=3, indep, err ϵ): $3\epsilon^2(1 - \epsilon) + \epsilon^3$.
- Bayes' classifier: $\hat{y} = \arg \max_k P(x | y = k)P(y = k)$.

Common pitfalls

- **Forgetting to standardise** for KNN / K-means / Ridge / Lasso / GD-with-momentum.
- **Forgetting Laplace smoothing** in Naive Bayes — one zero kills everything.
- **Using R^2** for model selection — always improves with more features; use adjusted or CV.
- **Confusing $\{0, 1\}$ and $\{-1, +1\}$ labels** — logistic uses $\{0, 1\}$, SVM uses $\{\pm 1\}$.
- **Treating CV folds as independent** in deriving SE — they share data.
- **Reading 'high CV error' as 'high bias'** — could be either; check train error too.
- **Forgetting $\alpha \geq 0$** for inequality constraints in Lagrangian. Equality multipliers are unrestricted.
- **Tuning on the test set** — touch it once at the end.

Time budget — exam day

- **5 min:** read every question, allocate by marks.
- **For derivations:** state assumption $\rightarrow$ write formula $\rightarrow$ substitute $\rightarrow$ simplify $\rightarrow$ boxed answer.
- **For numericals:** show formula + plug-in + compute + units.
- **For SVM / Lasso geometry questions:** SKETCH.
- **For boundary-case Q's:** check $\lambda = 0, \infty$ and $k = 1, n$.
- **Last 10 min:** re-read, check signs, check feasibility.
- **Never leave blank** — formula alone often gets partial credit.

01_Probability_Explainer

Pre-Mid-Sem 1: Probability

Foundation for everything that follows. MLE, Bayes' rule (used in GDA, Naive Bayes), bias-variance decomposition — all built on these basics.

1. Sample space, events, axioms

- **Sample space** Ω : set of all possible outcomes.
- **Event** $A \subseteq \Omega$: a subset of outcomes.
- **Probability** $P : \text{events} \rightarrow [0, 1]$ satisfying:
 1. $P(A) \geq 0$ for every event A .
 2. $P(\Omega) = 1$.
 3. **Countable additivity**: for disjoint $A_1, A_2, \dots$: $P(\bigcup_i A_i) = \sum_i P(A_i)$.

Consequences: $P(\emptyset) = 0$; $P(A^c) = 1 - P(A)$; for general A, B : $P(A \cup B) = P(A) + P(B) - P(A \cap B)$.

2. Conditional probability and Bayes' rule

Conditional probability:

$$P(A | B) = \frac{P(A \cap B)}{P(B)}, \quad P(B) > 0.$$

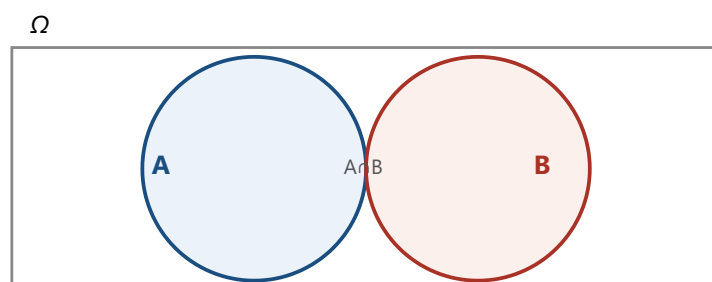
Multiplication rule: $P(A \cap B) = P(A | B) P(B) = P(B | A) P(A)$.

Law of total probability: for a partition $A_1, \dots, A_n$ of Ω :

$$P(B) = \sum_i P(B | A_i) P(A_i).$$

Bayes' rule (the formula that drives every generative classifier):

$$P(A | B) = \frac{P(B | A) P(A)}{P(B)}$$



$P(A|B) = P(A \cap B)/P(B)$: restrict to B , ask the fraction in A

3. Independence

A and B are **independent** if $P(A \cap B) = P(A)P(B)$, equivalently $P(A | B) = P(A)$. Knowing B doesn't change the probability of A .

Conditional independence: $A \perp B | C$ iff $P(A \cap B | C) = P(A | C)P(B | C)$. (Different from marginal — this is the Naive Bayes assumption.)

4. Random variables, expectation, variance

A **random variable** X maps outcomes to numbers. Discrete RVs have a probability mass function (PMF) $p(x) = P(X = x)$.

Expectation (mean): $E[X] = \sum_x x p(x)$. **Variance:** $\text{Var}(X) = E[(X - E[X])^2] = E[X^2] - (E[X])^2$.

Standard deviation: $\sigma(X) = \sqrt{\text{Var}(X)}$.

Linearity of expectation — ALWAYS holds

For **any** RVs X, Y (independent or not) and constants a, b, c :

$$E[aX + bY + c] = a E[X] + b E[Y] + c.$$

No independence assumption needed.

Variance addition — only when independent

For **independent** X, Y :

$$\text{Var}(X + Y) = \text{Var}(X) + \text{Var}(Y).$$

General: $\text{Var}(X + Y) = \text{Var}(X) + \text{Var}(Y) + 2 \text{Cov}(X, Y)$, where $\text{Cov}(X, Y) = E[(X - EX)(Y - EY)]$.

Scaling: $\text{Var}(aX) = a^2 \text{Var}(X)$.

5. Common distributions you must memorise

Distribution	PMF / parameters	$E[X]$	$\text{Var}(X)$
Bernoulli(p)	$X \in \{0, 1\}, P(X = 1) = p$	p	$p(1 - p)$
Binomial(n, p)	$X \sim \# \text{ successes in } n \text{ trials}$	np	$np(1 - p)$
Geometric(p)	$\# \text{ tosses until first success}$	$1/p$	$(1 - p)/p^2$
Uniform $\{1, \dots, n\}$	discrete uniform	$(n + 1)/2$	$(n^2 - 1)/12$
Fair die	Uniform $\{1, \dots, 6\}$	3.5	35/12
Normal $\mathcal{N}(\mu, \sigma^2)$	continuous bell	μ	σ^2

6. Worked sample problems

Problem 1 — Conditional probability

A box has 3 red and 2 blue marbles. You draw two without replacement. What's the probability the second is red given the first is red?

Solution. After drawing a red, 2 red and 2 blue remain. $P(2\text{nd red} \mid 1\text{st red}) = 2/4 = 0.5$.

Problem 2 — Bayes' rule

A test for a disease has $P(+ \mid \text{disease}) = 0.99$ (sensitivity), $P(+ \mid \text{no disease}) = 0.05$ (false positive). Disease prevalence is 0.01. You test positive. What's $P(\text{disease} \mid +)$?

Solution. Bayes: $P(D \mid +) = \frac{P(+ \mid D)P(D)}{P(+)}$.

$$P(+) = P(+ \mid D)P(D) + P(+ \mid D^c)P(D^c) = 0.99 \cdot 0.01 + 0.05 \cdot 0.99 = 0.0099 + 0.0495 = 0.0594.$$

$P(D \mid +) = 0.0099/0.0594 \approx 0.167$. **Only ~17%** even after a positive test — base rate matters.

Problem 3 — Expectation of die roll

A fair die is rolled. Let X be the value shown. Compute $E[X]$ and $\text{Var}(X)$.

Solution. $E[X] = (1 + 2 + 3 + 4 + 5 + 6)/6 = 3.5$. $E[X^2] = (1 + 4 + 9 + 16 + 25 + 36)/6 = 91/6$.
 $\text{Var}(X) = 91/6 - 12.25 = 35/12 \approx 2.92$.

Problem 4 — Two dice

Two fair dice rolled independently, sum is $S = X_1 + X_2$. Compute $E[S]$ and $\text{Var}(S)$.

Solution. $E[S] = 3.5 + 3.5 = 7$ (linearity). $\text{Var}(S) = 35/12 + 35/12 = 35/6 \approx 5.83$ (independence).

Problem 5 — Indicator trick

Roll 6 dice. What's the expected number of sixes?

Solution. Let $X_i = \mathbb{1}\{\text{die } i = 6\}$. $X = \sum X_i$. $E[X_i] = 1/6$. By linearity: $E[X] = 6 \cdot 1/6 = 1$. (Don't need independence!)

Problem 6 — Bernoulli/binomial connection

Show $X \sim \text{Binomial}(n, p)$ has $E[X] = np$ via indicators.

Solution. $X = \sum_{i=1}^n X_i$ where each $X_i \sim \text{Bernoulli}(p)$ counts whether trial i succeeded. $E[X_i] = p$, so $E[X] = np$.

Problem 7 — Conditional vs unconditional independence

Toss two fair coins. Let $A = \text{"first heads"}$, $B = \text{"second heads"}$, $C = \text{"the two coins show different faces"}$. Show A and B are independent but A and B are NOT independent given C .

Solution. Marginally: $P(A) = P(B) = 1/2$, $P(A \cap B) = 1/4$. Yes independent. Given C : $P(A \mid C) = 1/2$ (half of the discordant outcomes — HT, TH — have first heads). $P(B \mid C) = 1/2$. But $P(A \cap B \mid C) = 0$ (impossible, since C requires different faces). $0 \neq 1/2 \cdot 1/2$, **not** conditionally independent.

Problem 8 — Linearity holds without independence

X, Y both standard normal, but with $Y = -X$ (perfectly negatively correlated). What's $E[X + Y]$ and $\text{Var}(X + Y)$?

Solution. $E[X + Y] = E[X] + E[Y] = 0 + 0 = 0$ (linearity holds despite dependence).

$\text{Var}(X + Y) = \text{Var}(X - X) = \text{Var}(0) = 0$ (not $\text{Var}(X) + \text{Var}(Y) = 2$). Reason: variance addition requires independence; here $\text{Cov}(X, Y) = -1$.

7. Cheat-sheet take-aways

- $P(A | B) = P(A \cap B) / P(B)$. Bayes: $P(A | B) = P(B | A)P(A) / P(B)$.
- Total probability: $P(B) = \sum_i P(B | A_i)P(A_i)$.
- Independent $\Leftrightarrow P(A \cap B) = P(A)P(B)$.
- $E[aX + bY + c] = aE[X] + bE[Y] + c$ — always.
- $\text{Var}(X + Y) = \text{Var}(X) + \text{Var}(Y)$ — only if independent.
- $\text{Var}(X) = E[X^2] - (E[X])^2$.
- Bernoulli(p): $E = p$, $V = p(1 - p)$. Binomial(n, p): $E = np$, $V = np(1 - p)$.
- Use **indicator decomposition** for “expected count of X among N items” — linearity is your friend.

02_LinearAlgebra_Explainer

Pre-Mid-Sem 2: Linear Algebra

Linear regression, PCA, GDA, SVMs — everything runs on these. Master vectors, matrices, projections, eigen-decomposition.

1. Vectors and basic operations

A vector $v \in \mathbb{R}^n$ is an n -tuple of reals. Geometrically: an arrow from origin to a point.

Inner / dot product:

$$\langle u, v \rangle = u^\top v = \sum_i u_i v_i.$$

Properties: $\|v\| = \sqrt{v^\top v}$ (Euclidean norm). $\cos \theta = \frac{u^\top v}{\|u\|\|v\|}$ (angle).

Orthogonal: $u \perp v$ iff $u^\top v = 0$. **Orthonormal:** orthogonal AND unit-norm.

2. Linear independence, span, basis

Vectors $v_1, \dots, v_k$ are **linearly independent** if

$$\sum_i c_i v_i = 0 \quad \Rightarrow \quad c_i = 0 \quad \forall i.$$

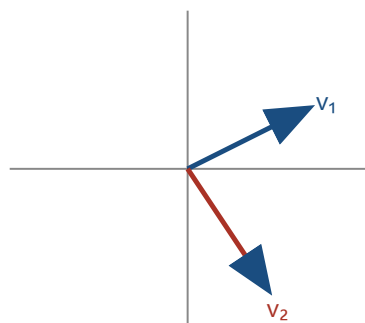
Equivalently: none of them can be written as a linear combination of the others.

Span: $\text{span}\{v_1, \dots, v_k\} = \{\sum_i c_i v_i : c_i \in \mathbb{R}\}$ — all linear combinations.

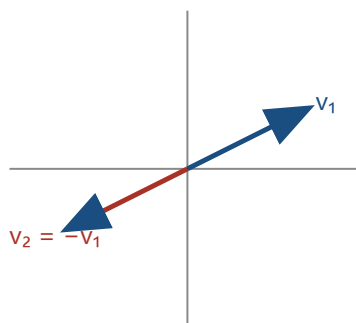
Basis: maximal independent set; spans the whole space. In $\mathbb{R}^n$: any basis has exactly n vectors.

Key fact: in $\mathbb{R}^n$, at most n linearly independent vectors. (Any $n + 1$ vectors are linearly dependent.)

independent



dependent



3. Projections (the heart of OLS)

Projection of y onto a unit vector u :

$$\text{proj}_u(y) = (u^\top y) u.$$

General (non-unit u): scale by $\frac{u^\top y}{u^\top u}$.

Projection onto the column space of $X \in \mathbb{R}^{n \times d}$ (assuming X has full column rank):

$$\hat{y} = X(X^\top X)^{-1}X^\top y.$$

The matrix $H = X(X^\top X)^{-1}X^\top$ is the **hat matrix / projection matrix**. Key properties: $H^2 = H$ (idempotent), $H^\top = H$ (symmetric).

This is why OLS is a projection: $\hat{y} = X\hat{\beta}$ minimises $\|X\hat{\beta} - y\|^2$, which is the closest point in $\text{col}(X)$ to y — exactly the projection.

4. Matrix operations

- $(AB)^\top = B^\top A^\top$
- $(AB)^{-1} = B^{-1}A^{-1}$ (when invertible)
- $\text{trace}(AB) = \text{trace}(BA)$ (cyclic)
- $\$(A) = \$ \# \text{ linearly independent columns} = \# \text{ linearly independent rows}.$
- **Rank-nullity:** $\$(A) + (A) = \$ \# \text{ columns of } A.$
- Square A invertible $\Leftrightarrow \det A \neq 0 \Leftrightarrow \text{rows linearly independent} \Leftrightarrow \text{rank} = \text{full}.$

5. Eigenvalues and eigenvectors

For a square matrix A , $v \neq 0$ is an **eigenvector** with **eigenvalue** λ if

$$Av = \lambda v.$$

Recipe: solve $\det(A - \lambda I) = 0$ (characteristic equation) for λ ; then plug back into $(A - \lambda I)v = 0$ to find v .

For $A = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$: $\lambda^2 - (a + d)\lambda + (ad - bc) = 0$. (Trace = $a + d = \sum \lambda_i$, $\det = ad - bc = \prod \lambda_i$.)

Symmetric matrices (i.e., $A = A^\top$) have especially nice eigendecompositions: - All eigenvalues real. - Eigenvectors for distinct eigenvalues are orthogonal. - $A = U\Lambda U^\top$ where U has orthonormal eigenvector columns and $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_n)$.

6. PSD matrices

A is **positive semidefinite (PSD)**, written $A \succeq 0$, if

$$x^\top A x \geq 0 \quad \forall x.$$

Equivalent: all eigenvalues ≥ 0 . **Positive definite (PD)**: > 0 . PSD matrices have non-negative trace, non-negative determinant.

Covariance matrices, $X^\top X$, and Σ in Gaussians are all PSD. **Convexity check**: f convex $\Leftrightarrow$ Hessian PSD.

7. Worked sample problems

Problem 1 — Linear independence by hand

Are $v_1 = (1, 2, 3)$, $v_2 = (2, 1, 0)$, $v_3 = (3, 3, 3)$ linearly independent?

Solution. Form A with these as columns. $\det(A) = 1(3 - 0) - 2(6 - 0) + 3(6 - 3) = 3 - 12 + 9 = 0$. $\det = 0 \Rightarrow$ dependent. Indeed, $v_3 = v_1 + v_2$.

Problem 2 — Projection

Project $y = (1, 2, 3)$ onto $u = (1, 0, 0)$.

Solution. u is already unit. $\text{proj}_u(y) = (u^\top y)u = 1 \cdot (1, 0, 0) = (1, 0, 0)$.

Problem 3 — Hat matrix idempotence

Verify $H = X(X^\top X)^{-1}X^\top$ satisfies $H^2 = H$.

Solution.

$$H^2 = X(X^\top X)^{-1}X^\top \cdot X(X^\top X)^{-1}X^\top = X(X^\top X)^{-1} \underbrace{(X^\top X)(X^\top X)^{-1}}_{=I} X^\top = X(X^\top X)^{-1}X^\top = H.$$

Problem 4 — Eigenvalues

Find eigenvalues of $A = \begin{pmatrix} 3 & 1 \\ 0 & 2 \end{pmatrix}$.

Solution. Triangular matrix $\Rightarrow$ eigenvalues are diagonal entries: $\lambda_1 = 3$, $\lambda_2 = 2$. Check:

$$\det(A - \lambda I) = (3 - \lambda)(2 - \lambda) = 0. \checkmark$$

Problem 5 — PSD check

Is $A = \begin{pmatrix} 2 & 1 \\ 1 & 3 \end{pmatrix}$ PSD?

Solution. Symmetric. Eigenvalues from $(2 - \lambda)(3 - \lambda) - 1 = \lambda^2 - 5\lambda + 5 = 0$: $\lambda = (5 \pm \sqrt{5})/2 > 0$. Both positive $\Rightarrow$ PD $\Rightarrow$ PSD. Shortcut: diagonal entries ≥ 0 and $\det = 5 \geq 0$.

Problem 6 — Geometric interpretation of OLS

In linear regression, $\hat{y} = Hy$ is the projection of y onto $\text{col}(X)$. What is the residual $e = y - \hat{y}$ orthogonal to?

Solution. $e \perp \text{col}(X)$. Indeed

$X^\top e = X^\top (y - Hy) = X^\top y - (X^\top X)(X^\top X)^{-1}X^\top y = X^\top y - X^\top y = 0$. This is the **normal equation**.

Problem 7 — Computing rank

For $A = \begin{pmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \end{pmatrix}$, find rank.

Solution. Row 2 = 2 × Row 1, so only 1 independent row. Rank = 1.

8. Cheat-sheet take-aways

- $\|v\| = \sqrt{v^\top v}$; $\cos \theta = (u^\top v) / (\|u\| \|v\|)$.
- Linear independence $\Leftrightarrow$ no $c \neq 0$ with $\sum c_i v_i = 0$.
- $\mathbb{R}^n$: max n independent vectors; basis has exactly n .
- Projection onto $\text{col}(X)$: $H = X(X^\top X)^{-1}X^\top$. $H^2 = H$, $H^\top = H$.
- Rank-nullity: $\text{rank}(A) + \text{nullity}(A) = n$ (cols).
- Eigenvalue: $\det(A - \lambda I) = 0$. Eigenvector: $(A - \lambda I)v = 0$.
- For 2×2 : trace = sum of λ , det = product of λ .
- Symmetric A : real eigenvalues, orthogonal eigenvectors, $A = U\Lambda U^\top$.
- PSD $\Leftrightarrow x^\top A x \geq 0 \forall x \Leftrightarrow$ all $\lambda \geq 0$.

03_LinearRegression_Explainer

Pre-Mid-Sem 3: Linear Regression (OLS) — Derivation, Geometry, Interpretation, Evaluation

Covers L3a (intro), L5 (interpreting OLS), L5b (Advertising case), L6 (OLS evaluation), L6b (Advertising revisited). The single most important topic — every classifier in the post-mid-sem syllabus is a variation of this.

1. The model

Given data $\{(x^{(i)}, y^{(i)})\}_{i=1}^n$, $x^{(i)} \in \mathbb{R}^d$ (or $\mathbb{R}^{d-1}$ + intercept). Linear regression assumes

$$y^{(i)} = \beta_0 + \beta_1 x_1^{(i)} + \cdots + \beta_{d-1} x_{d-1}^{(i)} + \varepsilon^{(i)} = x^{(i)\top} \beta + \varepsilon^{(i)},$$

where $\varepsilon^{(i)}$ are i.i.d. noise terms with $E[\varepsilon] = 0$, $\text{Var}(\varepsilon) = \sigma^2$.

In matrix form: $y = X\beta + \varepsilon$ with $X \in \mathbb{R}^{n \times d}$, $\beta \in \mathbb{R}^d$.

2. OLS — least squares estimate

Loss (residual sum of squares, RSS):

$$L(\beta) = \frac{1}{2} \|X\beta - y\|^2.$$

Setting $\nabla L = 0$:

$$\nabla L = X^\top(X\beta - y) = 0 \Rightarrow X^\top X\beta = X^\top y.$$

These are the **normal equations**. Assuming $X^\top X$ is invertible (i.e., X has full column rank):

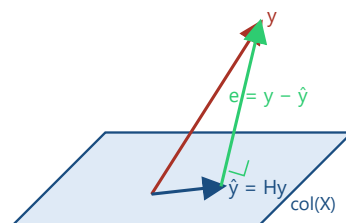
$$\hat{\beta} = (X^\top X)^{-1} X^\top y.$$

1-D special case ($d = 2$ with intercept): with $\bar{x}, \bar{y}$ the sample means,

$$\hat{\beta}_1 = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2}, \quad \hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}.$$

3. Geometric interpretation

$\hat{y} = X\hat{\beta} = X(X^\top X)^{-1} X^\top y = Hy$ — the projection of y onto $\text{col}(X)$.



$\hat{y}$ is the closest point in $\text{col}(X)$ to y ; residual $e \perp \text{col}(X)$

4. Statistical properties of OLS

Under Gauss-Markov assumptions ($E[\varepsilon | X] = 0$, $\text{Var}(\varepsilon | X) = \sigma^2 I$, X deterministic):

- $\hat{\beta}$ is **unbiased**: $E[\hat{\beta}] = \beta$.
- $\text{Var}(\hat{\beta}) = \sigma^2 (X^\top X)^{-1}$.
- **Gauss-Markov theorem**: OLS is **BLUE** — Best (lowest variance) Linear Unbiased Estimator.

Under the additional assumption that noise is Gaussian, OLS = MLE.

5. OLS evaluation

RSS, TSS, R^2

- **RSS** (residual sum of squares): $\sum (y_i - \hat{y}_i)^2$.
- **TSS** (total sum of squares): $\sum (y_i - \bar{y})^2$.
- **ESS** (explained): $\sum (\hat{y}_i - \bar{y})^2$. $\text{RSS} + \text{ESS} = \text{TSS}$.

$$R^2 = 1 - \frac{\text{RSS}}{\text{TSS}} = \frac{\text{ESS}}{\text{TSS}}.$$

Reads: "fraction of variance explained by the model." Ranges over $[0, 1]$ (for OLS with intercept). $R^2 = 1 \Rightarrow$ perfect fit; $R^2 = 0 \Rightarrow$ no better than predicting $\bar{y}$.

Variance estimate: $\hat{\sigma}^2 = \text{RSS}/(n - p)$ where p is the number of parameters (degrees of freedom adjustment).

t-statistic

$$t_j = \frac{\hat{\beta}_j}{\text{SE}(\hat{\beta}_j)}, \quad \text{SE}(\hat{\beta}_j) = \sqrt{\hat{\sigma}^2 [(X^\top X)^{-1}]_{jj}}.$$

Tests $H_0 : \beta_j = 0$. Large $|t_j| \Rightarrow$ reject $H_0 \Rightarrow$ coefficient is significant.

Bootstrap

Resample data with replacement, refit $\hat{\beta}$ many times. Empirical distribution of $\hat{\beta}$ approximates the sampling distribution. Used to estimate SE and confidence intervals when analytic formulas are hard or assumptions are doubtful.

6. Interpreting coefficients

$\hat{\beta}_j$ = predicted change in y per unit change in x_j , **holding all other features constant**.

Confounding pitfall (Advertising case study). TV, Radio, Newspaper budgets. Univariate regression on Newspaper alone shows positive coefficient (newspaper "works"). Multivariate regression with all three: Newspaper coefficient drops near 0. Reason: Newspaper budget correlates with Radio; once Radio is in the model, Newspaper has nothing left to explain.

Take-away: a coefficient's value depends on what other features are in the model. Univariate $\neq$ multivariate coefficients in general.

7. Worked sample problems

Problem 1 — OLS by hand (3 points)

Data: $(1, 2), (2, 3), (3, 5)$. Fit $y = \beta_0 + \beta_1 x$.

Solution. $\bar{x} = 2, \bar{y} = 10/3$.

$$\sum (x_i - \bar{x})(y_i - \bar{y}) = (-1)(2 - 10/3) + 0 + (1)(5 - 10/3) = (-1)(-4/3) + (5/3) = 4/3 + 5/3 = 3.$$

$$\sum (x_i - \bar{x})^2 = 1 + 0 + 1 = 2. \hat{\beta}_1 = 3/2 = 1.5. \hat{\beta}_0 = 10/3 - 1.5 \cdot 2 = 10/3 - 3 = 1/3 \approx 0.33. \text{Model: } \hat{y} = 0.33 + 1.5x.$$

Problem 2 — R^2 for the above

Solution. Predictions: $\hat{y} = (1.83, 3.33, 4.83)$. $RSS = (2 - 1.83)^2 + (3 - 3.33)^2 + (5 - 4.83)^2 = 0.029 + 0.109 + 0.029 = 0.167$.
 $TSS = (2 - 10/3)^2 + (3 - 10/3)^2 + (5 - 10/3)^2 = (4/3)^2 + (1/3)^2 + (5/3)^2 = 16/9 + 1/9 + 25/9 = 42/9 \approx 4.67$.
 $R^2 = 1 - 0.167/4.67 \approx 0.964$. **Excellent fit.**

Problem 3 — Normal equations

For data above, write the normal equation in matrix form.

Solution. $X = \begin{pmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{pmatrix}$, $y = (2, 3, 5)$. $X^\top X = \begin{pmatrix} 3 & 6 \\ 6 & 14 \end{pmatrix}$, $X^\top y = (10, 23)$. Normal eq: $\begin{pmatrix} 3 & 6 \\ 6 & 14 \end{pmatrix} \beta = (10, 23)$. Solve:
 $\det = 42 - 36 = 6$, $\hat{\beta} = \frac{1}{6} \begin{pmatrix} 14 & -6 \\ -6 & 3 \end{pmatrix} (10, 23) = \frac{1}{6} (140 - 138, -60 + 69) = (1/3, 3/2)$. ✓ matches Problem 1.

Problem 4 — Adding a feature

If you add a useless feature to a multivariate regression, what happens to RSS and to R^2 ?

Solution. RSS can never *increase* (adding columns to X can only enlarge $\text{col}(X)$, so the projection of y gets closer). Thus R^2 never decreases. This is why R^2 alone is a bad model-selection criterion — you'd always pick the biggest model. Use **adjusted R^2** or AIC/BIC.

Problem 5 — Geometric interpretation of normal equations

Show that the normal equations $X^\top (y - X\hat{\beta}) = 0$ are equivalent to saying the residual is orthogonal to every column of X .

Solution. $X^\top (y - X\hat{\beta})$ is the vector of dot products $\langle x_j, y - X\hat{\beta} \rangle$ for each column x_j of X . Setting it to zero means $\langle x_j, e \rangle = 0 \forall j$, i.e., residual $e \perp x_j$ for every j .

Problem 6 — Coefficient interpretation under confounding

You regress y (sales) on x_1 (TV budget) alone, get $\hat{\beta}_1 = 0.05$. Then regress on x_1 and x_2 (Radio), get $\hat{\beta}_1 = 0.046$, $\hat{\beta}_2 = 0.19$. Explain the small change in $\hat{\beta}_1$.

Solution. If TV and Radio budgets are weakly correlated, controlling for Radio barely changes the TV coefficient — TV's effect on sales doesn't go through Radio. (Compare: Newspaper coefficient collapsed because Newspaper was correlated with Radio; without Radio it absorbed Radio's effect; with Radio it has nothing left.)

Problem 7 — Why $\hat{\sigma}^2 = \text{RSS}/(n - p)$ and not RSS/n

Solution. We've fit p parameters to the data, so p of the n residuals are constrained ($X^\top e = 0$ gives p linear constraints). Only $n - p$ residuals are "free." Dividing by $n - p$ rather than n corrects for this and makes $\hat{\sigma}^2$ unbiased.

Problem 8 — Test the significance of a coefficient

OLS output: $\hat{\beta}_1 = 0.04$, $\text{SE} = 0.01$. Is the coefficient significantly different from zero (at the conventional 5% level)?

Solution. $t = 0.04/0.01 = 4$. For large n , $|t| > 2$ is roughly $p < 0.05$. With $|t| = 4$, $p \ll 0.001$. **Highly significant.** Reject H_0 .

8. Cheat-sheet take-aways

- $\hat{\beta} = (X^\top X)^{-1} X^\top y$. Geometric: projection of y onto $\text{col}(X)$.
- 1-D: $\hat{\beta}_1 = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2}$; $\hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}$.
- Normal equations: $X^\top X \beta = X^\top y \Leftrightarrow \text{residual} \perp \text{col}(X)$.
- $R^2 = 1 - \text{RSS}/\text{TSS}$. $\hat{\sigma}^2 = \text{RSS}/(n - p)$.
- $t_j = \hat{\beta}_j / \text{SE}(\hat{\beta}_j)$. $|t_j| > 2 \approx \text{significant at 5\%}$.
- OLS is **BLUE** under Gauss-Markov. Coefficient = effect of x_j on y holding others fixed; depends on the model.
- Adding any feature never increases RSS / never decreases R^2 .

04_KNN_BiasVariance_Validation_Explainer

Pre-Mid-Sem 4: KNN, Validation, Bias-Variance, Overfitting

Covers 3_KNN_Validation and 4_Bias_Variance_Trade-off_Overfitting_Regularization. Two distinct topics tied together by one question: *how well will my model generalise?*

Part A — K-Nearest Neighbours (KNN)

The algorithm — zero training

KNN is **non-parametric** and **lazy** — no parameters to fit, all the work happens at prediction time.

To predict for a new point x : 1. Compute distance from x to every training point. 2. Find the k closest training points. 3. **Classification**: majority vote of those k neighbours' labels. **Regression**: average of those k neighbours' targets.

Hyperparameter k

- $k = 1$: very flexible — every training point gets its own Voronoi region. Low bias, high variance (overfits).
- $k = N$: predict the majority class / global mean for every x . High bias, low variance (underfits).
- Sweet spot: chosen by cross-validation.

Curse of dimensionality

In high d , "nearest" neighbours stop being close. Distances concentrate (ratio of nearest to farthest $\rightarrow 1$). KNN loses meaning. Roughly: KNN works well when d is small or you have a *lot* of data.

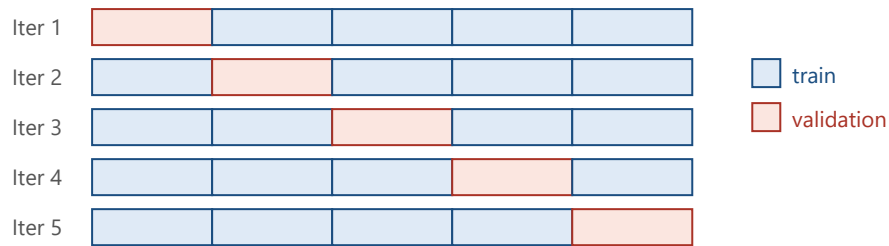
Part B — Train / Validation / Test split

A model's training error is **not** a good estimate of how it generalises — it always optimistically biased.

Standard practice: - **Training set (~60-80%)**: fit model parameters. - **Validation set (~10-20%)**: tune hyperparameters (e.g., k in KNN, λ in ridge). - **Test set (~10-20%)**: final, untouched evaluation. Touch it once.

K-fold cross-validation

Better use of limited data: 1. Split training data into K equal folds. 2. For each fold k : train on the other $K - 1$ folds, validate on fold k . 3. Report average validation error across folds.

5-fold cross-validation

Each fold serves as validation once; CV error = mean of the 5 validation errors.

Part C — Bias-Variance decomposition

For squared loss, the expected test error at a new point x_0 decomposes:

$$E[(y - \hat{f}(x_0))^2] = \underbrace{(E[\hat{f}(x_0)] - f(x_0))^2}_{\text{Bias}^2} + \underbrace{\text{Var}(\hat{f}(x_0))}_{\text{Variance}} + \underbrace{\sigma^2}_{\text{Irreducible noise}}.$$

- **Bias:** how far the average prediction (over training sets) is from the truth — error from oversimplifying.
- **Variance:** how much $\hat{f}(x_0)$ varies across different training sets — error from being sensitive to data.
- **Irreducible noise** σ^2 : unavoidable, doesn't depend on the model.

The trade-off

	High bias	High variance
Symptom	Underfits	Overfits
Train error	High	Low
Test error	High	High
Gap (train vs test)	Small	Large
Cure	More complex model, more features	Regularise, simpler model, more data, ensemble



Part D — Overfitting / underfitting

Overfitting: model captures noise rather than signal. Low training error, high test error. Often comes from: - Too many parameters / too flexible. - Too little data. - Training too long.

Underfitting: model too simple to capture pattern. Both errors high.

Diagnostics

- **Learning curves:** plot train and validation error vs training set size or epochs. Big train-val gap $\Rightarrow$ overfit. Both stuck high $\Rightarrow$ underfit.

Cures for overfitting

- **More training data.**
- **Regularisation** (L1/L2 — see topic 8).
- **Simpler model** (fewer features, smaller tree, smaller k in polynomial regression).
- **Early stopping** (in iterative methods).
- **Ensembling** (averages out variance).

Part E — Worked sample problems

Problem 1 — KNN by hand

Training: $A = (1, 1)$ class 0, $B = (2, 2)$ class 0, $C = (5, 5)$ class 1, $D = (6, 6)$ class 1. Predict the class of $x = (3, 3)$ with $k = 3$.

Solution. Euclidean distances squared: $d(x, A) = 8$, $d(x, B) = 2$, $d(x, C) = 8$, $d(x, D) = 18$. Three closest: B (class 0), A (class 0), C (class 1) — tie between A, C at $d^2 = 8$, break by including both? Take $k = 3$ exactly closest: B, A, C . Vote: $2 \times$ class 0, $1 \times$ class 1. Predict 0.

Problem 2 — Why $k = 1$ overfits

Argue why $k = 1$ KNN has training error zero but high test error.

Solution. For any training point $x^{(i)}$, the nearest training point to it is itself. Predicted label = its own label. Training error = 0. But the model has no smoothing — every training point dominates a tiny Voronoi region; tiny perturbations of test data fall into neighbours dominated by the wrong-class outlier. Maximum variance, minimum bias.

Problem 3 — Compute CV error

You run 5-fold CV with these per-fold errors: 0.18, 0.22, 0.20, 0.25, 0.19. Report CV error.

Solution. Mean = $(0.18 + 0.22 + 0.20 + 0.25 + 0.19)/5 = 1.04/5 = 0.208$.

Problem 4 — Bias-Variance accounting

A constant predictor $\hat{f}(x) = c$ has zero variance. Why is it usually a bad predictor?

Solution. It's high-bias: $E[\hat{f}(x_0)] = c$, so bias $= (c - f(x_0))^2$, which is large unless f is also nearly constant. The trade-off zeros out variance at the cost of huge bias.

Problem 5 — Identify the regime

Train error 0.02, test error 0.30. Diagnose.

Solution. Huge gap (0.28) $\Rightarrow$ **overfitting** (high variance). Cures: more data, regularisation, simpler model, dropout/ensembling.

Problem 6 — Identify the regime (2)

Train error 0.40, test error 0.42. Diagnose.

Solution. Both high, gap small $\Rightarrow$ **underfitting** (high bias). Cures: more features, more flexible model, longer training.

Problem 7 — KNN regression

Targets at the same 4 points are $y = (2, 3, 9, 11)$. With $k = 2$, predict for $x = (4, 4)$.

Solution. Closest two: $B = (2, 2)$ ($d^2 = 8$) and $C = (5, 5)$ ($d^2 = 2$). Average $= (3 + 9)/2 = 6$.

Problem 8 — Why does more data help variance but not bias?

Solution. Variance scales like $1/n$ — more data $\Rightarrow$ smaller fluctuation in $\hat{f}$. Bias depends on the *model class* — it persists even with infinite data if the true f isn't in the class. So adding data fixes variance, not bias.

Part F — Cheat-sheet take-aways

- **KNN**: lazy, non-parametric. Vote (classif) / average (regress) over k nearest neighbours. $k \uparrow \Rightarrow$ smoother (more bias, less variance).
- Train/Val/Test: validation tunes hyperparameters; test is for **final** evaluation only.
- **K-fold CV**: average validation error across K folds. Standard $K = 5, 10$.
- **Bias-Variance**: Test MSE = Bias² + Variance + σ^2 . U-shaped vs model complexity.
- **Overfit** (high variance): train low, test high, big gap. Fix: more data, regularise, simpler model.
- **Underfit** (high bias): both high, small gap. Fix: more flexible model, more features.
- Adding data shrinks variance but doesn't fix bias.

05_FeatureEngineering_ModelSelection_Explainer

Pre-Mid-Sem 5: Feature Engineering + Model Selection

Lecture 7. Garbage in, garbage out — feature choice often matters more than model choice. Then how to pick among candidate models.

1. Feature scaling — why and how

Different features on different scales (e.g., age 18-90, income 10k-500k) can wreck distance-based models (KNN, K-means), gradient-based optimisation (slow convergence on elongated valleys), and regularisation (penalises large-scale coefficients more aggressively for no good reason).

Standardisation (z-score): $\tilde{x}_j = (x_j - \mu_j) / \sigma_j$ — each feature becomes mean 0, std 1.

Min-max scaling: $\tilde{x}_j = (x_j - \min_j) / (\max_j - \min_j)$ — each feature in $[0, 1]$.

Always fit scaling on training data and apply the same parameters to test data. Never compute scaling stats over the full dataset (data leakage).

2. Polynomial features

Linear regression in raw x only fits straight lines / hyperplanes. To capture curves, **add transformed features**:

$$\phi(x) = (1, x, x^2, x^3, \dots, x^p).$$

Then fit linear regression on $\phi(x)$. The model is **linear in β** even though the relation y vs x is non-linear. All OLS theory still applies.

Multivariate: include cross terms $x_1 x_2, x_1^2 x_2$ etc — captures interactions.

Cost: parameter count explodes. With d features and degree p , polynomial features count is $\binom{d+p}{p}$. Overfitting becomes the main risk.



3. Categorical features

One-hot encode: a category with K values becomes $K - 1$ binary indicators (drop one to avoid perfect collinearity — that's the "reference" category, absorbed into the intercept).

4. Model selection — picking among candidates

You have several candidate models (different feature sets, different polynomial degrees, different regularisation strengths). Which one generalises best?

Adjusted R^2

$$R_{\text{adj}}^2 = 1 - \frac{\text{RSS}/(n - p - 1)}{\text{TSS}/(n - 1)}.$$

Penalises adding parameters that don't actually improve fit. Unlike R^2 , this can decrease when you add a useless feature.

AIC (Akaike Information Criterion)

$$\text{AIC} = 2p - 2 \log L = n \log(\text{RSS}/n) + 2p \quad (\text{for OLS}).$$

Lower is better. Penalty $2p$ — preferred for prediction.

BIC (Bayesian Information Criterion)

$$\text{BIC} = p \log n - 2 \log L = n \log(\text{RSS}/n) + p \log n.$$

Lower is better. Stronger penalty than AIC ($\log n > 2$ for $n \geq 8$). BIC prefers simpler models — preferred when you believe in a "true" sparse model.

Cross-validation

The most direct: estimate test error directly. Slower but assumption-free. Use when AIC/BIC assumptions (e.g., Gaussian errors) don't hold or when you don't trust them.

5. Subset selection

With p features, 2^p subsets. Brute force impractical for $p > \sim 30$. Heuristics:

Forward selection

1. Start with no features.
2. At each step, add the feature that most improves the criterion (AIC, BIC, validation error).
3. Stop when no addition helps.

Backward elimination

1. Start with all features.
2. At each step, drop the feature whose removal most improves the criterion.
3. Stop when no drop helps.

Both are greedy — not guaranteed optimal, but tractable. Backward fails when $p > n$ (can't fit full model); forward works there.

6. Worked sample problems

Problem 1 — Standardisation by hand

Three samples of a feature: (2, 4, 6). Standardise.

Solution. Mean = 4, std (population) = $\sqrt{((2-4)^2 + 0 + (6-4)^2)/3} = \sqrt{8/3} \approx 1.633$.
Standardised: $((2-4)/1.633, 0, (6-4)/1.633) = (-1.22, 0, 1.22)$.

Problem 2 — Effect of forgetting to scale

You train KNN with features (age in years, income in rupees). What's the practical effect of not scaling?

Solution. Income variations swamp age variations in Euclidean distance. The neighbourhood is effectively determined by income alone. KNN essentially ignores age — fixing this with standardisation often dramatically improves accuracy.

Problem 3 — Polynomial feature count

With $d = 3$ features and polynomial degree $p = 2$ (including all cross terms), how many features in $\phi(x)$?

Solution. Constant (1) + degree-1 (3) + degree-2 (3 squared + $\binom{3}{2} = 3$ cross terms) = $1 + 3 + 6 = 10$.
Formula: $\binom{d+p}{p} = \binom{5}{2} = 10$. ✓

Problem 4 — Adjusted R^2 vs R^2

Model A: 5 features, $R^2 = 0.81$. Model B: 10 features, $R^2 = 0.82$. $n = 100$. Which is preferred under adjusted R^2 ?

Solution. $R_{\text{adj}}^2(A) = 1 - (1 - 0.81) \frac{99}{94} = 1 - 0.19 \cdot 1.053 = 1 - 0.200 = 0.800$.

$R_{\text{adj}}^2(B) = 1 - (1 - 0.82) \frac{99}{89} = 1 - 0.18 \cdot 1.112 = 1 - 0.200 = 0.800$. Tie. Model A is simpler — prefer A by parsimony.

Problem 5 — Compare AIC for two models

$n = 100$. Model A: $p = 5$, $\text{RSS} = 30$. Model B: $p = 10$, $\text{RSS} = 25$. Which is preferred under AIC?

Solution. $\text{AIC}(A) = 100 \log(0.3) + 10 = -120.4 + 10 = -110.4$. $\text{AIC}(B) = 100 \log(0.25) + 20 = -138.6 + 20 = -118.6$. B has lower AIC $\Rightarrow$ prefer **B**.

Problem 6 — Same with BIC

Solution. $\log 100 \approx 4.6$. $\text{BIC}(A) = -120.4 + 5 \cdot 4.6 = -120.4 + 23 = -97.4$. $\text{BIC}(B) = -138.6 + 10 \cdot 4.6 = -138.6 + 46 = -92.6$. A has lower BIC $\Rightarrow$ prefer **A** (BIC penalty stronger).

Problem 7 — When forward selection fails

A feature is only useful in combination with another. Forward selection considers features one at a time. Will it find this feature?

Solution. Not necessarily — neither feature alone may improve the criterion, so neither gets added in step 1, and they never enter the model. This is the **interaction effect** problem. Lasso (topic 8) handles it better since it considers all features simultaneously, and best-subset (brute force) handles it exactly at exponential cost.

Problem 8 — Why one-hot drop one column

Why do we drop one column when one-hot encoding a categorical feature?

Solution. With all K indicators included, they sum to 1 for every row, so $\mathbf{1} = \sum_k \mathbf{1}_k$ — perfect collinearity with the intercept. $X^\top X$ becomes singular and OLS has no unique solution. Dropping one indicator removes the collinearity; the dropped category becomes the baseline against which others' coefficients are measured.

7. Cheat-sheet take-aways

- **Always scale** features for KNN/K-means/gradient methods/regularised regression.
- **Polynomial features** make linear regression non-linear in x but linear in β — keeps OLS theory.
- **Adjusted R^2** : penalises useless features (unlike plain R^2).
- **AIC** = $n \log(\text{RSS}/n) + 2p$ — lighter penalty, predictive.
- **BIC** = $n \log(\text{RSS}/n) + p \log n$ — stronger penalty, parsimony.
- **Forward**: start empty, add best; **Backward**: start full, drop worst.
- **CV** is the safest model-selection criterion when you can afford it.
- One-hot: drop one column to avoid singular $X^\top X$.

06_ConvexOptimization_Explainer

L9–L10: Convex Optimization — Concept Clearing + Sample Problems

Companion to the handwritten lecture L9_10-ConvexOptimization (1).pdf . Reads top-to-bottom in the same order as the lecture, but expands each idea with intuition, a clean derivation, and at the end a set of fully worked sample problems.

Roadmap (what the lecture is doing, and why)

The whole lecture is building a **toolkit** so that next week, when SVMs appear, you already know how to:

1. Recognise that the SVM problem is a *constrained convex* problem.
2. Write down its **Lagrangian**.
3. Take the **dual**.
4. Use **KKT conditions** to read off the optimal w from the dual variables α_i , and to see why only support vectors matter.

So everything in L9–L10 is foundational. The four sections of the lecture are:

Section	What it gives you
Convex sets	The shape of <i>feasible regions</i> — where solutions live
Convex functions	The shape of <i>objectives</i> — what we minimise
Constrained convex opt	The standard form (P) : min convex f over convex set
Lagrangian dual + KKT	The machinery to actually solve (P) — and the optimality certificate

1. Convex sets

Definition

A set $\mathcal{X} \subseteq \mathbb{R}^n$ is **convex** if for every $x, y \in \mathcal{X}$ and every $\alpha \in [0, 1]$:

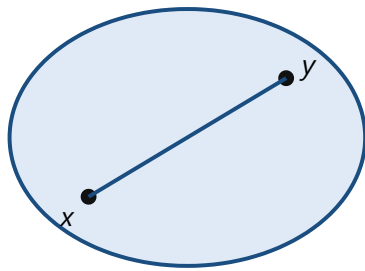
$$\alpha x + (1 - \alpha)y \in \mathcal{X}$$

The point $\alpha x + (1 - \alpha)y$ is called a **convex combination** of x and y . As α slides from 0 to 1, it traces the line segment from y to x . So the definition says:

The line segment between any two points of $\mathcal{X}$ lies entirely inside $\mathcal{X}$.

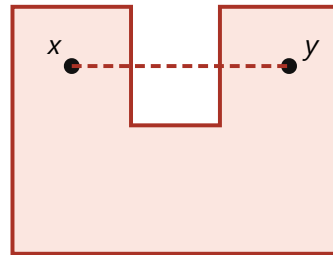
Intuition

- A filled disk in $\mathbb{R}^2$: convex (any chord stays inside).
- A crescent moon: **not** convex (a chord can cut through the empty bite).
- $\mathbb{R}^n$ itself: convex.
- The empty set and a single point: convex (vacuously / trivially).



CONVEX

segment stays inside the set



NOT CONVEX

dashed chord exits the set

The flagship example — polyhedral set

Let A be an $m \times n$ matrix and $b \in \mathbb{R}^m$. The set

$$\mathcal{X} = \{x \in \mathbb{R}^n : Ax \leq b\}$$

(where $\leq$ is **pointwise** — every coordinate of Ax is $\leq$ the corresponding coordinate of b) is convex. This set is called a **polyhedron** or **polyhedral set**.

Why it's convex. Take x, y with $Ax \leq b$ and $Ay \leq b$. For $\alpha \in [0, 1]$,

$$A(\alpha x + (1 - \alpha)y) = \alpha Ax + (1 - \alpha)Ay \leq \alpha b + (1 - \alpha)b = b.$$

So the convex combination is also feasible. Done.

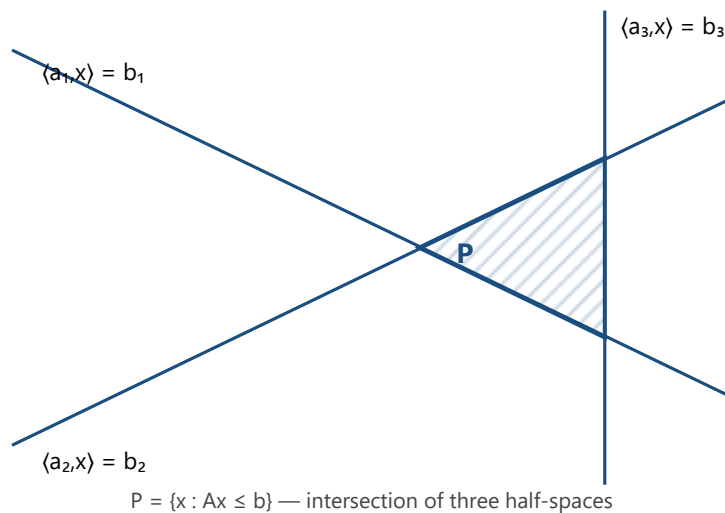
Halfspaces, hyperplanes, and “intersection of halfspaces”

Let $a_i^\top$ denote the i -th row of A . Then $Ax \leq b$ unpacks as

$$\langle a_1, x \rangle \leq b_1, \quad \langle a_2, x \rangle \leq b_2, \quad \dots, \quad \langle a_m, x \rangle \leq b_m.$$

Each line $\langle a_i, x \rangle = b_i$ is a **hyperplane** (an $(n - 1)$ -dimensional flat slice). Each inequality $\langle a_i, x \rangle \leq b_i$ defines a **halfspace** (everything on one side of that hyperplane, including the hyperplane itself).

A polyhedron is exactly **the intersection of finitely many halfspaces**. This generalises: *intersection of any family of convex sets is convex* (the chord stays in each set, hence in the intersection).



2. Convex functions

Definition

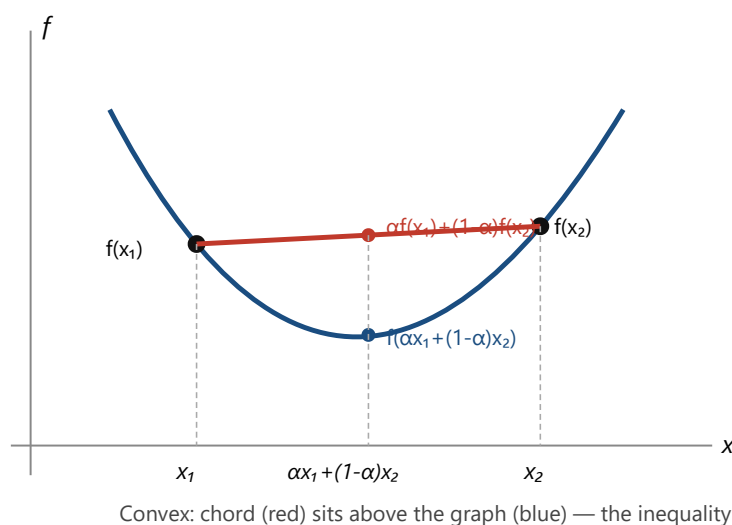
Let $\mathcal{X} \subseteq \mathbb{R}^n$ be convex. A function $f : \mathcal{X} \rightarrow \mathbb{R}$ is **convex** if for all $x_1, x_2 \in \mathcal{X}$ and $\alpha \in [0, 1]$:

$$f(\alpha x_1 + (1 - \alpha)x_2) \leq \alpha f(x_1) + (1 - \alpha)f(x_2).$$

Intuition — chord above graph

The left side is the function value at the convex combination of the inputs. The right side is the convex combination of the function values — geometrically, the chord from $(x_1, f(x_1))$ to $(x_2, f(x_2))$ evaluated at the same horizontal position.

A function is convex iff its graph lies on or below every chord.



Mirror image: f is **concave** iff $-f$ is convex, i.e., the graph lies on or above every chord. (Maximising a concave function = minimising a convex function — same problem.)

Why we care

Convex functions have **no spurious local minima**. Every local minimum is also a global minimum. That is *the* reason the entire ML pipeline (linear regression, ridge, SVM, logistic regression) is built around convex losses — local search (gradient descent) finds the global answer.

Recall: gradient, Hessian, PSD

For $f : \mathcal{X} \rightarrow \mathbb{R}$ with $\mathcal{X} \subseteq \mathbb{R}^n$, at a point x :

- **Gradient** (a vector in $\mathbb{R}^n$):

$$\nabla f(x) = \begin{pmatrix} \partial f / \partial x_1 \\ \vdots \\ \partial f / \partial x_n \end{pmatrix}.$$

- **Hessian** (a symmetric $n \times n$ matrix; symmetric because $\partial^2 f / \partial x_i \partial x_j = \partial^2 f / \partial x_j \partial x_i$ for C^2 functions):

$$[\nabla^2 f(x)]_{ij} = \frac{\partial^2 f}{\partial x_i \partial x_j}.$$

- **Positive semidefinite (PSD)**: a symmetric matrix J is PSD, written $J \succeq 0$, iff

$$x^\top J x \geq 0 \quad \forall x \in \mathbb{R}^n.$$

(Equivalent: all eigenvalues are ≥ 0 .)

Two clean characterisations of convexity

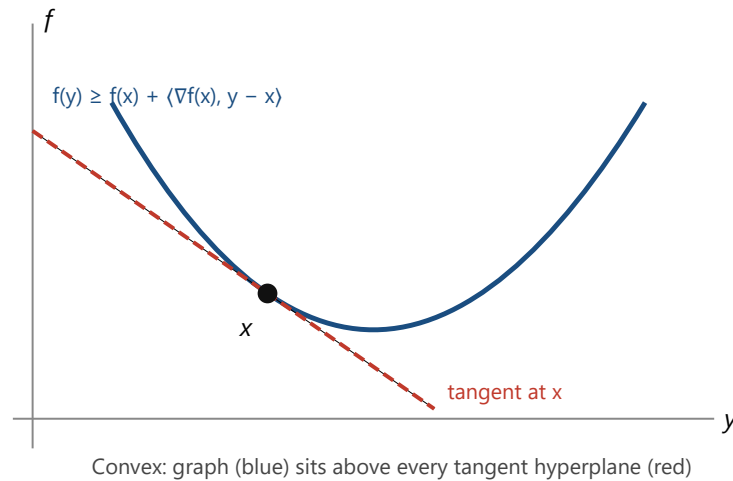
Both assume $\mathcal{X}$ is convex.

First-order (needs f differentiable). f is convex iff

$$f(y) \geq f(x) + \langle \nabla f(x), y - x \rangle \quad \forall x, y \in \mathcal{X}.$$

The right side is the **tangent hyperplane** at x , viewed as a linear function of y . So:

The graph of a convex function sits above every one of its tangent hyperplanes.



This is also why a stationary point ($\nabla f(x) = 0$) of a convex function is a global minimum: the tangent hyperplane is flat, and the graph sits above it.

Second-order (needs f twice differentiable). f is convex iff

$$\nabla^2 f(x) \succeq 0 \quad \forall x \in \mathcal{X}.$$

This is the test you actually use in problems: compute the Hessian, check PSD.

Two flagship convex functions (from the lecture)

Quadratic form with PSD matrix. Let Σ be symmetric PSD. Then $f(x) = x^\top \Sigma x$ is convex.

Proof via Hessian. Expanding, $\nabla f(x) = 2\Sigma x$ and $\nabla^2 f(x) = 2\Sigma \succeq 0$. Done.

Any norm. $f(x) = \|x\|$ is convex.

Proof via definition (the lecture's proof). Using positive homogeneity ($\|\alpha v\| = |\alpha| \|v\|$, and $\alpha, 1 - \alpha \geq 0$) and the triangle inequality:

$$\|\alpha x_1 + (1 - \alpha)x_2\| \leq \|\alpha x_1\| + \|(1 - \alpha)x_2\| = \alpha\|x_1\| + (1 - \alpha)\|x_2\|.$$

This proof never used differentiability — handy, because $\|\cdot\|$ is not differentiable at 0.

3. Constrained convex optimization — the primal (P)

The lecture writes the standard primal form:

$$(P) \quad \min_{x \in \mathcal{X}} f(x) \quad \text{s.t.} \quad Ax \leq b$$

with:

- f a **convex** objective,
- $\mathcal{X} \subseteq \mathbb{R}^n$ a **convex** subset (often just $\mathbb{R}^n$),
- $A \in \mathbb{R}^{m \times n}$, $b \in \mathbb{R}^m$.

The full **feasible region** is $\mathcal{X} \cap \{x : Ax \leq b\}$ — the intersection of two convex sets, hence convex. So (P) is: *minimise a convex function over a convex set*. We denote the **optimal value** by p^* .

Why this matters: convex objective + convex feasible region $\Rightarrow$ local min = global min, gradient descent works, and (as we'll see) a strong dual exists.

4. The Lagrangian and the dual (D)

Building the Lagrangian — the “penalty” idea

The constraint $Ax \leq b$ is equivalent to the m separate constraints $\langle a_i, x \rangle - b_i \leq 0$. Suppose we “soften” each constraint by attaching a non-negative price $\alpha_i \geq 0$ for violating it. The Lagrangian is:

$$L(x, \alpha) = f(x) + \sum_{i=1}^m \alpha_i (\langle a_i, x \rangle - b_i) = f(x) + \alpha^\top (Ax - b).$$

Read this as: *minimise the objective, but pay α_i units of penalty for every unit of constraint i that you violate*. The α_i are called **Lagrange multipliers** (one per inequality).

Key observation: if x is feasible ($Ax \leq b$) and $\alpha \geq 0$, then $\alpha^\top (Ax - b) \leq 0$, so $L(x, \alpha) \leq f(x)$.

Minimising L over *all* x (feasible or not) gives a **lower bound** on the optimal value p^* . This is the core duality idea.

The dual objective

For each fixed vector of multipliers $\alpha \geq 0$, define

$$F(\alpha) = \inf_{x \in \mathcal{X}} L(x, \alpha).$$

$F(\alpha)$ is the best (lowest) bound we can squeeze out at this choice of prices. Two facts:

1. $F(\alpha) \leq p^*$ for every $\alpha \geq 0$ (**weak duality**).
2. F is **concave in α** — because $L(x, \alpha)$ is linear (and hence concave) in α for each fixed x , and the pointwise infimum of a family of concave functions is concave.

The dual problem

We now hunt for the *tightest* lower bound by maximising F :

$$(D) \quad \max_{\alpha \in \mathbb{R}_+^m} F(\alpha).$$

This is *maximising a concave function over a convex set* — itself a convex optimisation problem. Its variables are exactly the Lagrange multipliers. Denote the optimal value d^* .

Strong duality

For convex problems “well enough behaved” (this lecture treats it as a fact for our setting):

$$p^* = d^*.$$

Solving (D) is equivalent to solving (P) , but the dual is often easier or more insightful — this is exactly what makes the SVM dual useful (it's where kernels enter).

5. KKT conditions — the optimality certificate

Theorem. x^* solves (P) iff there exists $\alpha^* \in \mathbb{R}_+^m$ such that:

KKT condition	Meaning
$\nabla_x L(x^*, \alpha^*) = \nabla f(x^*) + \sum_i \alpha_i^* a_i = 0$	x^* minimises L over x at fixed α^* — stationarity
$Ax^* \leq b$	x^* is feasible for (P) — primal feasibility
$\alpha^* \geq 0$	Multipliers non-negative — dual feasibility
$\alpha_i^* (\langle a_i, x^* \rangle - b_i) = 0 \ \forall i$	Complementary slackness

And in that case α^* solves (D) .

Reading complementary slackness

For each constraint i , *at the optimum*, at least one of the following holds:

- $\alpha_i^* = 0$ — the constraint is “inactive” (you weren’t paying for it), or
- $\langle a_i, x^* \rangle = b_i$ — the constraint is **active / tight** (the optimum sits exactly on the boundary).

This is exactly why, later, **only SVM support vectors get nonzero α_i** : non-support training points satisfy their margin constraint strictly, so by complementary slackness their multiplier is 0.

How you actually use KKT

Two main solve patterns:

- **Forward:** guess which constraints are active. For each guess, solve stationarity + active equalities, then check the rest (multipliers ≥ 0 , inactive constraints satisfied).
- **Dual route:** form L , eliminate x to get $F(\alpha)$, maximise $F(\alpha)$ over $\alpha \geq 0$ to find α^* , then recover x^* from stationarity (this is the route the lecture takes in its worked example).

6. The lecture’s worked example — every step

$$\min x_1^2 + 4x_2^2 \quad \text{s.t.} \quad x_1 + x_2 \leq -1, \quad (x_1, x_2) \in \mathbb{R}^2.$$

This is a convex problem: $f(x) = x_1^2 + 4x_2^2$ has Hessian $\text{diag}(2, 8) \succ 0$ (so even strictly convex), and the constraint defines a halfspace.

Step 1 — write the Lagrangian

One inequality, so one multiplier $\alpha \geq 0$. Move “ $x_1 + x_2 \leq -1$ ” into standard form “ $x_1 + x_2 + 1 \leq 0$ ” (rewrite $Ax \leq b$ with $b = -1$, so $Ax - b = x_1 + x_2 + 1$):

$$L(x_1, x_2, \alpha) = x_1^2 + 4x_2^2 + \alpha(x_1 + x_2 + 1).$$

Step 2 — minimise L over x at fixed α (get the dual objective)

Differentiate L and set to zero:

$$\frac{\partial L}{\partial x_1} = 2x_1 + \alpha = 0 \Rightarrow x_1^* = -\frac{\alpha}{2},$$

$$\frac{\partial L}{\partial x_2} = 8x_2 + \alpha = 0 \Rightarrow x_2^* = -\frac{\alpha}{8}.$$

Substitute back:

$$F(\alpha) = \left(-\frac{\alpha}{2}\right)^2 + 4\left(-\frac{\alpha}{8}\right)^2 + \alpha\left(-\frac{\alpha}{2} - \frac{\alpha}{8} + 1\right) = \frac{\alpha^2}{4} + \frac{\alpha^2}{16} - \frac{\alpha^2}{2} - \frac{\alpha^2}{8} + \alpha.$$

Find a common denominator (16): $\frac{4\alpha^2 + \alpha^2 - 8\alpha^2 - 2\alpha^2}{16} + \alpha = -\frac{5\alpha^2}{16} + \alpha$. So

$$F(\alpha) = \alpha\left(1 - \frac{5\alpha}{16}\right).$$

This is a concave parabola in α — exactly as theory predicts.

Step 3 — solve the dual

$$(D) \quad \max_{\alpha \geq 0} \alpha\left(1 - \frac{5\alpha}{16}\right).$$

Differentiate w.r.t. α : $1 - \frac{10\alpha}{16} = 0 \Rightarrow \alpha^* = \frac{16}{10} = \frac{8}{5}$. This is ≥ 0 , so the constraint $\alpha \geq 0$ is inactive — good.

Step 4 — recover the primal optimum via KKT (stationarity)

$$x_1^* = -\frac{\alpha^*}{2} = -\frac{8/5}{2} = -\frac{4}{5}, \quad x_2^* = -\frac{\alpha^*}{8} = -\frac{8/5}{8} = -\frac{1}{5}.$$

Step 5 — sanity-check all four KKT conditions

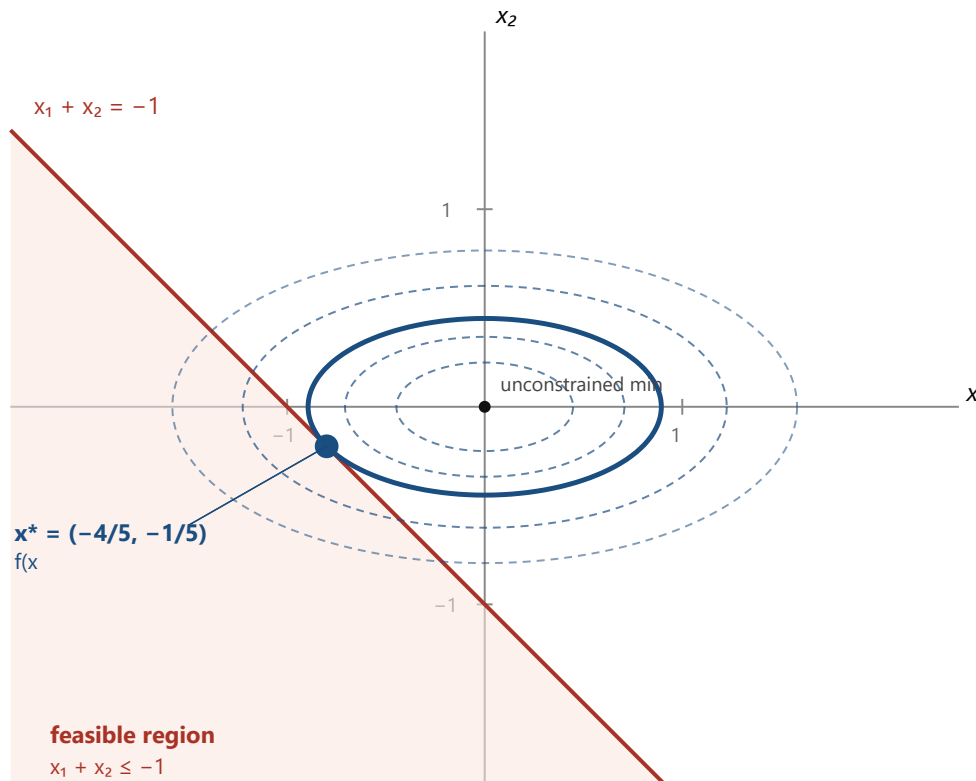
- **Stationarity:** $\nabla L = (2x_1^* + \alpha^*, 8x_2^* + \alpha^*) = (-\frac{8}{5} + \frac{8}{5}, -\frac{8}{5} + \frac{8}{5}) = (0, 0)$. ✓
- **Primal feasibility:** $x_1^* + x_2^* = -\frac{4}{5} - \frac{1}{5} = -1 \leq -1$. ✓ (Active with equality.)
- **Dual feasibility:** $\alpha^* = \frac{8}{5} \geq 0$. ✓
- **Complementary slackness:** $\alpha^*(x_1^* + x_2^* + 1) = \frac{8}{5} \cdot 0 = 0$. ✓ — consistent: $\alpha^* > 0$ exactly because the constraint is *active*.

Optimal primal value: $p^* = \left(-\frac{4}{5}\right)^2 + 4\left(-\frac{1}{5}\right)^2 = \frac{16}{25} + \frac{4}{25} = \frac{20}{25} = \frac{4}{5}$. Optimal dual value: $d^* = F\left(\frac{8}{5}\right) = \frac{8}{5}\left(1 - \frac{5 \cdot 8/5}{16}\right) = \frac{8}{5}\left(1 - \frac{1}{2}\right) = \frac{4}{5}$. ✓ $p^* = d^*$.

Geometric picture

Below: level curves of $f(x_1, x_2) = x_1^2 + 4x_2^2$ are concentric **ellipses** (squashed along x_2 because of the 4). The constraint $x_1 + x_2 \leq -1$ is a halfplane (shaded red). The optimum sits at $\left(-\frac{4}{5}, -\frac{1}{5}\right)$, where the

level curve $f = \frac{4}{5}$ kisses the constraint boundary — any tighter ellipse misses the feasible region, any looser one over-shoots.



$p = 4/5 = p$ $f = 4/5$ (tangent ellipse)

Why this picture matches KKT. At the optimum, the ellipse and the constraint line touch tangentially. Geometrically, “tangent” means the gradient of f at x^* (perpendicular to the level curve) is parallel to the gradient of the constraint $a_1 = (1, 1)$ (perpendicular to the line). That’s *exactly* the stationarity condition $\nabla f(x^*) + \alpha^* a_1 = 0$ — the multiplier $\alpha^* = \frac{8}{5}$ is the proportionality constant.

7. Sample problems (with full solutions)

Problem 1 — convex set check

Is $S = \{(x_1, x_2) \in \mathbb{R}^2 : x_1 x_2 \geq 1, x_1 > 0\}$ convex? What about $S \cup S'$, where $S' = \{(x_1, x_2) : x_1 x_2 \geq 1, x_1 < 0\}$?

Solution. S is convex. Take $(a, b), (c, d) \in S$. With $\lambda \in [0, 1]$, the convex combination has first coordinate $\lambda a + (1 - \lambda)c > 0$. For the product:

$$(\lambda a + (1 - \lambda)c)(\lambda b + (1 - \lambda)d) \geq (a^\lambda c^{1-\lambda})(b^\lambda d^{1-\lambda}) = (ab)^\lambda (cd)^{1-\lambda} \geq 1^\lambda \cdot 1^{1-\lambda} = 1,$$

using AM-GM ($\lambda u + (1 - \lambda)v \geq u^\lambda v^{1-\lambda}$ for $u, v > 0$).

$S \cup S'$ is **not** convex: $(1, 1) \in S$ and $(-1, -1) \in S'$, but their midpoint $(0, 0)$ has product $0 < 1$, so it’s not in the union. **Take-away:** union of convex sets is generally not convex.

Problem 2 — convex function via Hessian

Is $f(x_1, x_2) = x_1^2 + 2x_2^2 - 2x_1x_2$ convex on $\mathbb{R}^2$?

Solution. Compute partials: $\partial f / \partial x_1 = 2x_1 - 2x_2$, $\partial f / \partial x_2 = 4x_2 - 2x_1$. Hessian:

$$\nabla^2 f = \begin{pmatrix} 2 & -2 \\ -2 & 4 \end{pmatrix}.$$

PSD check: symmetric. Eigenvalues: trace = 6, det = 8 - 4 = 4 > 0. Both eigenvalues positive $\Rightarrow$ PSD (in fact PD) $\Rightarrow f$ is **strictly convex**.

Alternative shortcut for 2×2 : PSD iff both diagonal entries ≥ 0 and det ≥ 0 . Here 2, 4 ≥ 0 and det = 4 ≥ 0 . $\checkmark$

Problem 3 — convexity of a non-quadratic

Show $f(x) = \log(1 + e^x)$ on $\mathbb{R}$ (the **softplus**) is convex. *Why care?* It's the loss in logistic regression.

Solution. $f'(x) = \frac{e^x}{1+e^x} = \sigma(x)$ (the sigmoid). Then $f''(x) = \sigma(x)(1 - \sigma(x))$. Since $\sigma(x) \in (0, 1)$, we have $f''(x) > 0$ for all x . So f is strictly convex.

Problem 4 — Lagrangian / KKT, equality variant

$$\min \frac{1}{2}(x_1^2 + x_2^2) \quad \text{s.t.} \quad x_1 + x_2 = 1.$$

Solution. Equality constraint, so the multiplier μ is unrestricted in sign (not ≥ 0). Lagrangian:

$$L = \frac{1}{2}(x_1^2 + x_2^2) + \mu(x_1 + x_2 - 1).$$

Stationarity: $x_1 + \mu = 0$, $x_2 + \mu = 0 \Rightarrow x_1 = x_2 = -\mu$. Substitute in constraint:

$-2\mu = 1 \Rightarrow \mu = -\frac{1}{2}$, so $x_1^* = x_2^* = \frac{1}{2}$. Optimum: $p^* = \frac{1}{2}(\frac{1}{4} + \frac{1}{4}) = \frac{1}{4}$. Geometric reading: closest point in the line $x_1 + x_2 = 1$ to the origin under Euclidean distance.

Problem 5 — KKT with an inactive constraint

$$\min (x_1 - 1)^2 + (x_2 - 1)^2 \quad \text{s.t.} \quad x_1 + x_2 \leq 5.$$

Solution. The unconstrained minimum is $(1, 1)$, which satisfies $1 + 1 = 2 \leq 5$. So the constraint is

inactive. By complementary slackness, $\alpha^* = 0$. KKT all hold: stationarity

$\Rightarrow 2(x - 1) + 0 = 0 \Rightarrow x^* = (1, 1)$; primal feasibility $\checkmark$; dual feasibility $\checkmark$; CS $\checkmark$. **Lesson:** when the unconstrained minimum is feasible, the optimum is the unconstrained one and all multipliers are zero.

Problem 6 — full KKT walkthrough on a 2-D problem

$$\min x_1^2 + x_2^2 \quad \text{s.t.} \quad x_1 + 2x_2 \leq -2.$$

Solution. Lagrangian: $L = x_1^2 + x_2^2 + \alpha(x_1 + 2x_2 + 2)$, $\alpha \geq 0$. Stationarity: $2x_1 + \alpha = 0$,

$2x_2 + 2\alpha = 0 \Rightarrow x_1 = -\alpha/2$, $x_2 = -\alpha$. Substitute into L to get the dual:

$$F(\alpha) = (\alpha/2)^2 + \alpha^2 + \alpha(-\alpha/2 - 2\alpha + 2) = \frac{\alpha^2}{4} + \alpha^2 - \frac{\alpha^2}{2} - 2\alpha^2 + 2\alpha = -\frac{5\alpha^2}{4} + 2\alpha.$$

Maximise over $\alpha \geq 0$: $F'(\alpha) = -\frac{5\alpha}{2} + 2 = 0 \Rightarrow \alpha^* = \frac{4}{5} \geq 0 \checkmark$.

Recover: $x_1^* = -\frac{2}{5}$, $x_2^* = -\frac{4}{5}$. Sanity: constraint value $-\frac{2}{5} + 2(-\frac{4}{5}) = -\frac{2}{5} - \frac{8}{5} = -2 \checkmark$ (active, as expected since $\alpha^* > 0$). Primal value $p^* = \frac{4}{25} + \frac{16}{25} = \frac{20}{25} = \frac{4}{5}$. Dual value $d^* = F(\frac{4}{5}) = -\frac{5}{4} \cdot \frac{16}{25} + 2 \cdot \frac{4}{5} = -\frac{4}{5} + \frac{8}{5} = \frac{4}{5} \checkmark$.

Problem 7 — interpreting KKT directly (no dual)

$$\min x_1^2 + x_2^2 \quad \text{s.t.} \quad x_1 \geq 1, x_2 \geq 2.$$

Solution. Rewrite in $\leq$ form: $-x_1 + 1 \leq 0$ and $-x_2 + 2 \leq 0$, so $a_1 = (-1, 0)$, $b_1 = -1$; $a_2 = (0, -1)$, $b_2 = -2$. Stationarity: $\nabla f + \alpha_1 a_1 + \alpha_2 a_2 = 0 \Rightarrow (2x_1 - \alpha_1, 2x_2 - \alpha_2) = 0$, so $x_1 = \alpha_1/2$, $x_2 = \alpha_2/2$.

The unconstrained min $(0, 0)$ violates both constraints, so guess both are active: $x_1^* = 1 \Rightarrow \alpha_1^* = 2$; $x_2^* = 2 \Rightarrow \alpha_2^* = 4$. Both multipliers $\geq 0 \checkmark$ — KKT satisfied. Optimum $p^* = 1 + 4 = 5$.

Problem 8 — convexity of a maximum

Show that if f_1, f_2 are convex on $\mathbb{R}^n$, then $g(x) = \max(f_1(x), f_2(x))$ is convex.

Solution. For $\alpha \in [0, 1]$ and any x, y :

$$g(\alpha x + (1 - \alpha)y) = \max(f_1(\alpha x + (1 - \alpha)y), f_2(\alpha x + (1 - \alpha)y)).$$

Both arguments of $\max$ are $\leq \alpha f_i(x) + (1 - \alpha)f_i(y) \leq \alpha g(x) + (1 - \alpha)g(y)$ (since $f_i \leq g$). The max of two things, each $\leq C$, is $\leq C$. Done. **Take-away:** the hinge loss $\max(0, 1 - y w^\top x)$ used in SVM is convex (max of two affine functions — both affine functions are convex).

8. What to memorise for the exam

- Convex set definition (line segment stays inside) and the polyhedral example.
- Convex function definition (chord above graph) + both characterisations:
 $f(y) \geq f(x) + \nabla f(x)^\top (y - x)$ and $\nabla^2 f \succeq 0$.
- Standard primal form (P) and that intersection of convex sets is convex.
- Lagrangian $L(x, \alpha) = f(x) + \alpha^\top (Ax - b)$ with $\alpha \geq 0$.
- Dual $F(\alpha) = \inf_x L(x, \alpha)$ is concave; $(D) = \max_{\alpha \geq 0} F(\alpha)$.
- The four **KKT conditions**: stationarity, primal feasibility, dual feasibility, complementary slackness.
- Strong duality: $p^* = d^*$ for our convex setting.
- Solve sequence: build $L \rightarrow$ eliminate $x \rightarrow$ maximise $F(\alpha) \rightarrow$ recover x^* via stationarity $\rightarrow$ verify all four KKT conditions.

9. Common pitfalls

- **Forgetting $\alpha \geq 0$ for inequality constraints.** For *equality* constraints the multiplier is unrestricted; for ≤ 0 inequalities (standard form) it's ≥ 0 .

- **Standard form sign mistakes.** Always convert to $g_i(x) \leq 0$ before writing $L = f + \sum \alpha_i g_i$. Otherwise the sign of α_i flips.
- **Confusing convex with concave.** $-f$ flips the inequality. The dual $F(\alpha)$ is concave, so we *maximise*, not minimise.
- **Skipping the feasibility check.** Always verify $Ax^* \leq b$ at the end — it's free credit on an exam and catches sign errors.
- **Assuming all constraints are active.** If the unconstrained minimum is already feasible, every multiplier is 0 and the constraints are inactive. Guess actives carefully.

07_SVM_Explainer

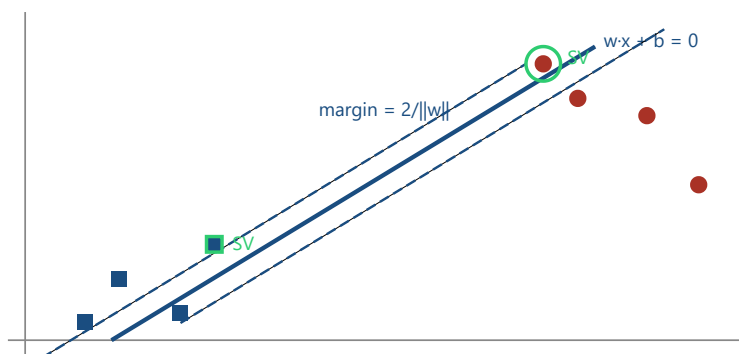
Pre-Mid-Sem 7: Support Vector Machines (Hard, Soft, Kernel)

Lectures 11, 11-12, 13. The single largest topic — combines convex optimisation, duality, KKT, geometry, and the kernel trick.

1. Hard-margin SVM — the maximum-margin classifier

Binary classification with labels $y \in \{-1, +1\}$ (not $\{0, 1\}$ — sign convention matters here). Assume data is **linearly separable**: there exists a hyperplane $w^\top x + b = 0$ such that $y^{(i)}(w^\top x^{(i)} + b) > 0$ for all training samples.

Among all separating hyperplanes, SVM picks the one with the **largest margin** — the largest distance from the hyperplane to the nearest training point on each side.



Distance from a point to a hyperplane

The signed distance from x to the hyperplane $w^\top x + b = 0$ is $(w^\top x + b)/\|w\|$.

The hard-margin primal

Scale (w, b) so that the closest training points sit on $w^\top x + b = \pm 1$. Then the margin (distance between $+1$ and -1 hyperplanes) is $2/\|w\|$.

Maximise margin $\iff$ **minimise** $\|w\|$:

$$\min_{w,b} \frac{1}{2} \|w\|^2 \quad \text{s.t.} \quad y^{(i)}(w^\top x^{(i)} + b) \geq 1 \quad \forall i.$$

Convex (quadratic objective, linear constraints).

The dual

Lagrangian:

$$L(w, b, \alpha) = \frac{1}{2} \|w\|^2 - \sum_i \alpha_i [y^{(i)}(w^\top x^{(i)} + b) - 1], \quad \alpha_i \geq 0.$$

Stationarity in w and b :

$$w^* = \sum_i \alpha_i^* y^{(i)} x^{(i)}, \quad \sum_i \alpha_i^* y^{(i)} = 0.$$

Plug back to get the dual (no w , no b , just α):

$$\max_{\alpha \geq 0} \sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y^{(i)} y^{(j)} \langle x^{(i)}, x^{(j)} \rangle \quad \text{s.t.} \quad \sum_i \alpha_i y^{(i)} = 0.$$

Support vectors and KKT

By complementary slackness: - $\alpha_i^* > 0 \Rightarrow$ constraint active $\Rightarrow y^{(i)}(w^{*\top} x^{(i)} + b^*) = 1 \Rightarrow x^{(i)}$ is on the margin boundary.

Support vector. - $\alpha_i^* = 0 \Rightarrow$ constraint slack $\Rightarrow x^{(i)}$ is strictly outside the margin and contributes nothing.

The final decision rule depends **only on the support vectors** — typically a small fraction of training data. This sparsity is one of SVM's beautiful properties.

2. Soft-margin SVM

Real data isn't perfectly separable. Introduce **slack variables** $\xi_i \geq 0$:

$$\min_{w,b,\xi} \frac{1}{2} \|w\|^2 + C \sum_i \xi_i \quad \text{s.t.} \quad y^{(i)}(w^\top x^{(i)} + b) \geq 1 - \xi_i, \quad \xi_i \geq 0.$$

ξ_i = how much sample i violates the margin (0 if correctly margin-classified, > 0 otherwise). C = cost of violations.

- **C large:** heavy penalty on errors $\rightarrow$ narrow margin, low bias, high variance.
- **C small:** tolerates errors $\rightarrow$ wide margin, more bias, lower variance.

Soft-margin dual

$$\max_{0 \leq \alpha_i \leq C} \sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y^{(i)} y^{(j)} \langle x^{(i)}, x^{(j)} \rangle \quad \text{s.t.} \quad \sum_i \alpha_i y^{(i)} = 0.$$

Only difference from hard-margin dual: α_i is **bounded above by C** . (Hard margin = $C = \infty$.)

Interpreting the dual variables in soft-margin

α_i value	ξ_i	Geometric meaning
$\alpha_i = 0$	$\xi_i = 0$	Outside margin, correctly classified — not a SV
$0 < \alpha_i < C$	$\xi_i = 0$	On the margin boundary — "margin SV"
$\alpha_i = C$	$\xi_i > 0$	Inside margin or misclassified — "non-margin SV"

3. Kernel SVM — non-linear boundaries for free

The kernel trick

Notice both duals depend on data **only through inner products** $\langle x^{(i)}, x^{(j)} \rangle$. So if we map $x \rightarrow \phi(x)$ into some higher-dimensional space and only need $\langle \phi(x^{(i)}), \phi(x^{(j)}) \rangle$, we can **replace this dot product with a kernel** $K(x^{(i)}, x^{(j)})$ that computes it directly — without ever materialising ϕ .

A kernel is **valid** (i.e., corresponds to *some* feature map) iff the Gram matrix $[K(x^{(i)}, x^{(j)})]$ is PSD for every dataset (**Mercer's theorem**).

Common kernels

Kernel	Formula	Feature space
Linear	$K(x, z) = \langle x, z \rangle$	x itself
Polynomial	$K(x, z) = (\langle x, z \rangle + c)^d$	All polynomial features up to degree d
RBF / Gaussian	$K(x, z) = \exp(-\gamma \ x - z\ ^2)$	Infinite-dimensional
Sigmoid	$\tanh(\kappa \langle x, z \rangle + c)$	(Not always valid)

The RBF kernel is the default workhorse — extremely flexible, but tune γ carefully (high $\gamma \rightarrow$ overfits; low $\gamma \rightarrow$ underfits).

Decision rule with kernels

Final classifier:

$$\hat{y}(x) = \text{sign} \left(\sum_{i: \alpha_i^* > 0} \alpha_i^* y^{(i)} K(x^{(i)}, x) + b^* \right).$$

You don't need w explicitly — just the support vectors, their α_i^* , and the kernel.

4. Worked sample problems

Problem 1 — Hard-margin SVM on 2 points

Two points: $(1, 0)$ with $y = +1$ and $(-1, 0)$ with $y = -1$. Find the maximum-margin hyperplane.

Solution. By symmetry, the decision boundary is $x_1 = 0$, i.e., $w = (1, 0)$, $b = 0$. Margin = $2/\|w\| = 2$. Both points are support vectors. Constraints $y^{(i)}(w^\top x^{(i)} + b) = 1$ satisfied: $+1 \cdot (1) = 1$, $-1 \cdot (-1) = 1$. ✓

Problem 2 — Why scale $\|w\| = 1$ smallest such that closest points are at ± 1 ?

Why does this rescaling not change the geometry?

Solution. The hyperplane equation $w^\top x + b = 0$ is invariant under $(w, b) \rightarrow (\lambda w, \lambda b)$ for any $\lambda > 0$ — same set of points. So we can fix the scale however we like. The convention $\min_i |w^\top x^{(i)} + b| = 1$ makes the algebra clean and lets us write the constraints as $y^{(i)}(w^\top x^{(i)} + b) \geq 1$.

Problem 3 — Compute w^* from α^*

For a 3-point problem with $\alpha^* = (2, 0, 2)$, $y = (+1, +1, -1)$, $x^{(1)} = (1, 1)$, $x^{(2)} = (5, 5)$, $x^{(3)} = (-1, -1)$, find w^* .

Solution. $w^* = \sum_i \alpha_i^* y^{(i)} x^{(i)} = 2(1)(1, 1) + 0 + 2(-1)(-1, -1) = (2, 2) + (2, 2) = (4, 4)$.

Support vectors: $x^{(1)}$ and $x^{(3)}$ (positive α); $x^{(2)}$ is not a SV.

Problem 4 — Interpret a soft-margin table

i	α_i^*	ξ_i	Description
1	0	0	?
2	$0.4C$	0	?
3	C	0.3	?
4	C	1.5	?

Solution. 1. Not a SV (outside margin). 2. Margin SV (on boundary). 3. Inside margin, **correctly** classified (since $\xi_i \in (0, 1)$). 4. **Misclassified** ($\xi_i > 1$).

Problem 5 — Verify a kernel is valid

Is $K(x, z) = (1 + x^\top z)^2$ a valid kernel for $x, z \in \mathbb{R}^2$?

Solution. Expand:

$(1 + x_1 z_1 + x_2 z_2)^2 = 1 + 2(x_1 z_1 + x_2 z_2) + (x_1 z_1 + x_2 z_2)^2 = 1 + 2x_1 z_1 + 2x_2 z_2 + x_1^2 z_1^2 + 2x_1 x_2 z_1 z_2 + x_2^2 z_2^2$. This equals $\langle \phi(x), \phi(z) \rangle$ where $\phi(x) = (1, \sqrt{2}x_1, \sqrt{2}x_2, x_1^2, \sqrt{2}x_1 x_2, x_2^2)$. Explicit feature map exists $\Rightarrow$ valid kernel.

Problem 6 — XOR with kernels

XOR data: $(0, 0) \rightarrow -1, (0, 1) \rightarrow +1, (1, 0) \rightarrow +1, (1, 1) \rightarrow -1$. Not linearly separable in $\mathbb{R}^2$. Show that with feature map $\phi(x) = (x_1, x_2, x_1 x_2)$, it becomes linearly separable.

Solution. In ϕ -space: $(0, 0, 0) \rightarrow -1, (0, 1, 0) \rightarrow +1, (1, 0, 0) \rightarrow +1, (1, 1, 1) \rightarrow -1$. Try $w = (1, 1, -2), b = -0.5$: dot products $-0.5, 0.5, 0.5, -0.5$ — exactly correct signs. Separable in ϕ -space, and the corresponding kernel $K(x, z) = x_1 z_1 + x_2 z_2 + x_1 x_2 z_1 z_2$ captures this without ever computing ϕ .

Problem 7 — Effect of C

You train soft-margin SVM with $C = 100$ and get train accuracy 100%, test accuracy 70%. Diagnose and suggest fix.

Solution. Huge gap $\Rightarrow$ overfitting (high variance). With $C = 100$, the model heavily penalises violations and contorts the boundary. Decrease C (say to 1 or 0.1) to widen the margin and improve generalisation.

Problem 8 — Why are SVMs sparse?

In contrast to logistic regression, where every training point contributes to the gradient and hence to the final w , SVM's w depends only on support vectors. Why?

Solution. Because of the **hinge-loss** structure: a non-support-vector $x^{(i)}$ has $y^{(i)}(w^\top x^{(i)} + b) > 1$, which makes its constraint inactive. Complementary slackness forces $\alpha_i = 0$, so it drops out of $w^* = \sum_i \alpha_i y^{(i)} x^{(i)}$. Logistic loss is smooth and never reaches zero, so every point always contributes.

5. Cheat-sheet take-aways

- **Hard-margin primal:** $\min \frac{1}{2} \|w\|^2$ s.t. $y^{(i)}(w^\top x^{(i)} + b) \geq 1$.
- **Soft-margin primal:** add $C \sum_i \xi_i$ and slack $\xi_i \geq 0$ in constraint.
- **Margin** $= 2/\|w\|$. **Maximise margin** $\Leftrightarrow$ **minimise** $\|w\|^2$.
- **Dual:** $\max_\alpha \sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y^{(i)} y^{(j)} \langle x^{(i)}, x^{(j)} \rangle$ s.t. $\sum_i \alpha_i y^{(i)} = 0, 0 \leq \alpha_i \leq C$ (soft) or $\alpha_i \geq 0$ (hard).
- $w^* = \sum_i \alpha_i^* y^{(i)} x^{(i)}$. Support vectors: $\alpha_i^* > 0$.
- **Support vector types** (soft margin): $0 < \alpha_i < C \Rightarrow$ on margin; $\alpha_i = C \Rightarrow$ inside margin / misclassified.
- **Kernel trick:** replace $\langle x^{(i)}, x^{(j)} \rangle$ by $K(x^{(i)}, x^{(j)})$ to learn non-linear boundaries without ever materialising ϕ .
- **Common kernels:** Linear, Polynomial ($\langle x, z \rangle + c$)^d, RBF $\exp(-\gamma \|x - z\|^2)$.
- **Mercer:** kernel valid iff Gram matrix is PSD for every dataset.

08_Shrinkage_RidgeLasso_Explainer

Pre-Mid-Sem 8: Shrinkage Methods — Ridge + Lasso

Lecture 14. When OLS overfits or breaks (e.g., $X^\top X$ singular), shrinkage methods pull coefficients toward zero by adding a penalty.

1. Why shrink?

OLS minimises RSS — unbiased but can be high-variance, especially when:

- p is close to n (many features, few samples).
- Features are correlated (multicollinearity $\rightarrow X^\top X$ near-singular $\rightarrow \hat{\beta}$ unstable).
- $p > n$: $X^\top X$ is singular, OLS doesn't even have a unique solution.

Shrinkage trades a little bias for a lot of variance reduction — overall test error often drops.

2. Ridge regression — L2 penalty

$$\hat{\beta}^{\text{ridge}} = \arg \min_{\beta} \|y - X\beta\|^2 + \lambda \|\beta\|_2^2,$$

with $\lambda \geq 0$ a hyperparameter.

Closed form

Setting gradient to zero:

$$\nabla = -2X^\top(y - X\beta) + 2\lambda\beta = 0 \Rightarrow (X^\top X + \lambda I)\beta = X^\top y.$$

$$\hat{\beta}^{\text{ridge}} = (X^\top X + \lambda I)^{-1} X^\top y.$$

Even when $X^\top X$ is singular, $X^\top X + \lambda I$ is invertible for any $\lambda > 0$ (we've shifted all eigenvalues up by λ). **Ridge gives a unique solution even when $p > n$.**

Effect on coefficients

Compared to OLS:

- All coefficients shrunk toward zero (not necessarily to zero).
- As $\lambda \rightarrow 0$: ridge $\rightarrow$ OLS.
- As $\lambda \rightarrow \infty$: $\hat{\beta} \rightarrow 0$.
- **Standardise features first** — otherwise the penalty unfairly punishes large-scale variables.

Bias-variance trade-off

$\hat{\beta}^{\text{ridge}}$ is **biased** (toward 0) but has lower variance than OLS. For appropriate λ , the MSE (bias² + var) is lower than OLS's.

3. Lasso regression — L1 penalty

$$\hat{\beta}^{\text{lasso}} = \arg \min_{\beta} \|y - X\beta\|^2 + \lambda \|\beta\|_1,$$

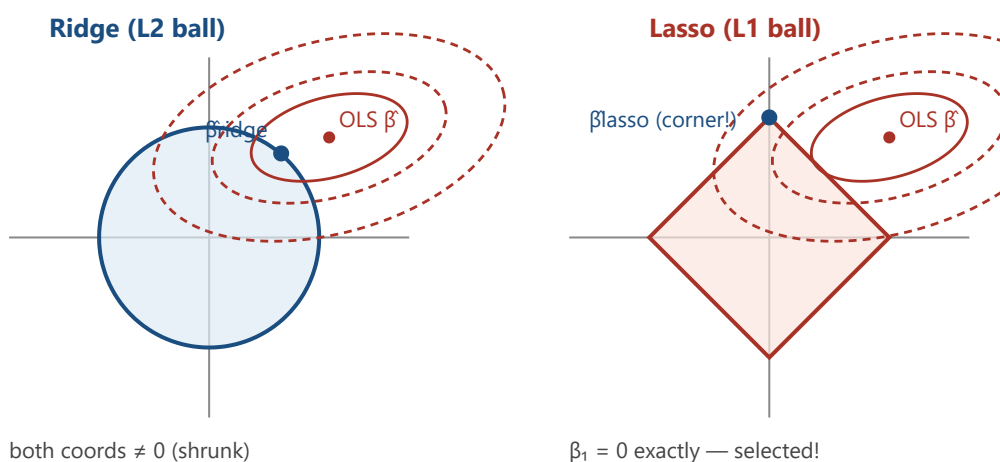
with $\|\beta\|_1 = \sum_j |\beta_j|$.

No closed form (L1 is not differentiable at zero), but the problem is convex — solve by coordinate descent or LARS.

Why Lasso produces sparse solutions

Reformulate as constrained: $\min \|y - X\beta\|^2$ s.t. $\|\beta\|_1 \leq t$ (some $t \leftrightarrow \lambda$). The L1 ball has **corners** at the coordinate axes. The OLS loss is convex; its level curves are ellipses. The optimum is where the ellipse first touches the constraint region. For L1, this is almost always at a **corner** — where one or more coordinates equal zero exactly.

For Ridge: the L2 ball is round (no corners); the touch point generically has all coordinates non-zero (just shrunk).



Lasso does automatic feature selection. Ridge shrinks but never zeros out.

4. Comparison

	OLS	Ridge	Lasso
Penalty	none	$\lambda \ \beta\ _2^2$	$\lambda \ \beta\ _1$
Closed form	$(X^\top X)^{-1} X^\top y$	$(X^\top X + \lambda I)^{-1} X^\top y$	None
Unique when $p > n$?	No	Yes	Yes
Feature selection?	No	No	Yes (sparse)
Bias / variance	Unbiased / high var	Some bias / less var	Some bias / less var
Stable under correlation?	No	Yes (averages correlated features)	Picks one, zeros others

Elastic Net combines both: $\lambda_1 \|\beta\|_1 + \lambda_2 \|\beta\|_2^2$ — handles correlation better than pure Lasso.

5. Choosing λ

Cross-validation. Try a grid of λ values, pick the one with lowest CV error. Often: - λ chosen to minimise CV error: best predictive performance. - λ_{1SE} : largest λ within 1 standard error of the minimum — simpler / more shrunk model with comparable CV error.

6. Worked sample problems

Problem 1 — Ridge by hand (1D)

Data: $X = (1, 2, 3)^\top$, $y = (2, 3, 5)^\top$. No intercept. Compute $\hat{\beta}^{\text{ridge}}$ for $\lambda = 1$.

Solution. $X^\top X = 1 + 4 + 9 = 14$. $X^\top y = 1 \cdot 2 + 2 \cdot 3 + 3 \cdot 5 = 2 + 6 + 15 = 23$. With $\lambda = 1$: $\hat{\beta} = 23/(14 + 1) = 23/15 \approx 1.533$. Compare to OLS: $23/14 \approx 1.643$. Ridge shrinks slightly.

Problem 2 — Ridge with $p > n$

You have $p = 100$ features, $n = 50$ samples. OLS fails — why? Does ridge work?

Solution. $X^\top X$ is 100×100 but has rank at most $n = 50$ (rank of X). So it's singular; $(X^\top X)^{-1}$ doesn't exist. Ridge adds λI to the diagonal, making it positive definite. Yes, ridge handles this.

Problem 3 — Why ridge is biased

Show: in the simple 1D no-intercept case, $E[\hat{\beta}^{\text{ridge}}] = \frac{X^\top X}{X^\top X + \lambda} \beta$.

Solution. $\hat{\beta}^{\text{ridge}} = X^\top y / (X^\top X + \lambda)$. With $y = X\beta + \varepsilon$ and $E[\varepsilon] = 0$: $E[X^\top y] = X^\top X\beta$. So $E[\hat{\beta}^{\text{ridge}}] = X^\top X\beta / (X^\top X + \lambda) = \frac{1}{1 + \lambda/(X^\top X)} \beta < \beta$. Shrinks below true value $\Rightarrow$ biased.

Problem 4 — Why Lasso zeros out

In 1D no-intercept, Lasso solves $\min(y - \beta x)^2 + \lambda|\beta|$. Show that for small $|x^\top y|$, the optimum is $\hat{\beta} = 0$.

Solution. Treat $\beta > 0$ and $\beta < 0$ separately. For $\beta > 0$: gradient is $-2x^\top(y - x\beta) + \lambda$. Set to zero: $\hat{\beta} = (2x^\top y - \lambda)/(2x^\top x)$. Positive only if $2x^\top y > \lambda$. Similarly for $\beta < 0$: need $2x^\top y < -\lambda$. If $|2x^\top y| < \lambda$, neither branch is feasible, optimum is $\hat{\beta} = 0$. **This is exactly soft-thresholding.**

Problem 5 — Compare Ridge & Lasso shapes

The constraint region of Lasso (L1 ball) has corners on the axes; Ridge's L2 ball is round. Why does this geometrically yield sparsity for Lasso?

Solution. The unconstrained loss has elliptical level curves. The constrained optimum is where the smallest level curve touches the constraint set. Ridge's circular boundary is smooth and tangent generically at non-axis points; coefficients all non-zero. Lasso's L1 ball has corners on the axes — the contact point is very often AT a corner, especially in high dimensions, forcing that coordinate to be exactly zero.

Problem 6 — Should you always standardise?

For OLS, standardising features doesn't change predictions, only the scale of coefficients. For Ridge and Lasso, standardising is essential — why?

Solution. OLS minimises RSS, which is invariant to feature scaling. Ridge/Lasso add a coefficient penalty that **is not** invariant — a feature on a scale 1000x larger has 1000x larger natural coefficient, so $\lambda\|\beta\|$ penalises it disproportionately. Standardise (mean 0, std 1) so that penalty is fair across features.

Problem 7 — Effect of correlated features

Two highly correlated features x_1, x_2 in a regression. OLS often gives $\hat{\beta}_1, \hat{\beta}_2$ with huge magnitudes and opposite signs (unstable). What does Ridge do? What does Lasso do?

Solution. Ridge **averages**: spreads weight across correlated features, gives a smaller, more stable pair of coefficients. Lasso typically picks **one** and zeros the other — sparser but more arbitrary. Elastic Net is a hybrid that often outperforms either.

Problem 8 — Choosing λ

You see this CV curve: CV error decreases from $\lambda = 0$, hits min at $\lambda^* = 0.1$, then increases. Where should you set λ ?

Solution. Pick $\lambda^* = 0.1$ for best predictive performance, or use the **1-SE rule** and pick the largest λ within 1 standard error of the minimum CV error for a simpler model that performs comparably.

7. Cheat-sheet take-aways

- **Ridge:** $\min \|y - X\beta\|^2 + \lambda\|\beta\|_2^2$. Closed form: $(X^\top X + \lambda I)^{-1} X^\top y$. **Unique even when $p > n$.** Coefficients shrunk, never zero.
- **Lasso:** $\min \|y - X\beta\|^2 + \lambda\|\beta\|_1$. No closed form. **Sparse solutions** — automatic feature selection.
- **Why Lasso is sparse:** L1 ball has corners on the axes; touch points often at corners.
- **Trade:** bias for variance. Both reduce variance vs OLS.
- **Standardise features** before applying — otherwise penalty is unfair.
- **Correlated features:** Ridge averages; Lasso picks one.
- **Choose λ by CV.** 1-SE rule for parsimony.
- Elastic Net: $\lambda_1\|\beta\|_1 + \lambda_2\|\beta\|_2^2$ — best of both for correlated features.

01_GradientDescent_LogisticRegression_Explainer

Post-Mid-Sem 1: Gradient Descent Variants + Logistic Regression

Lectures 15 (GD/SGD/Mini-batch) + 16 (Logistic Regression). These two go together because **logistic regression is trained by gradient descent** — there is no closed form like OLS.

Part A — Gradient Descent variants (L15)

The setup

We minimise a differentiable loss $L : \mathbb{R}^d \rightarrow \mathbb{R}$ over parameters θ :

$$\theta^* = \arg \min_{\theta} L(\theta).$$

The **gradient** $\nabla_{\theta} L$ is the vector of partial derivatives — it points in the direction of steepest *ascent*, so to *descend* we step the other way:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} L(\theta)$$

$\eta > 0$ is the **learning rate / step size**.

Linear regression as the running example

Data: $X \in \mathbb{R}^{n \times d}$ (each row a sample), $y \in \mathbb{R}^n$.

$$L(\theta) = \frac{1}{2n} \|X\theta - y\|^2, \quad \nabla_{\theta} L(\theta) = \frac{1}{n} X^{\top} (X\theta - y).$$

The $\frac{1}{2n}$ is a convenience constant (the $\frac{1}{2}$ cancels the chain-rule factor of 2; the $\frac{1}{n}$ averages over samples).

Batch GD update:

$$\theta \leftarrow \theta - \frac{\eta}{n} X^{\top} (X\theta - y).$$

Cost per step: $X\theta$ is $O(nd)$; the residual is $O(n)$; multiplying by $X^{\top}$ is $O(nd)$. **Total $O(nd)$ per step.**

That sounds fine until you plug in numbers:

Setting	n	d	Ops/step
Housing	10^7	50	$5 \cdot 10^8$
Image processing	10^6	10^5	10^{11}

For large n , batch GD is **too slow** and the memory to hold X in RAM can be prohibitive. This motivates the cheaper variants below.

Stochastic Gradient Descent (SGD)

Idea. At each step, pick one sample i uniformly at random and use the gradient of just *its* loss term:

$$L^{(i)}(\theta) = \frac{1}{2}(\hat{y}^{(i)} - y^{(i)})^2, \quad \nabla_{\theta} L^{(i)}(\theta) = (\hat{y}^{(i)} - y^{(i)}) x^{(i)}.$$

$$\theta \leftarrow \theta - \eta (\hat{y}^{(i)} - y^{(i)}) x^{(i)}$$

Cost per step: $O(d)$ — independent of n . We can take n such cheap steps in the same time one batch step takes.

Why does it work? Because i is drawn uniformly from $\{1, \dots, n\}$,

$$\mathbb{E}_i[\nabla_{\theta} L^{(i)}(\theta)] = \frac{1}{n} \sum_{i=1}^n \nabla_{\theta} L^{(i)}(\theta) = \nabla_{\theta} L(\theta).$$

So **on average, the SGD direction is the true gradient direction**. SGD is an *unbiased* estimator of batch GD. The trade-off is variance: each step is noisy, so trajectories zig-zag.

Decaying learning rate. Because the per-step noise doesn't vanish at the optimum, we damp it with a schedule like $\eta_t = 1/\sqrt{t}$ (decays slowly enough to still converge, fast enough to kill variance).

Algorithm (SGD with shuffling, one epoch = one pass over data): 1. Initialise θ . 2. For each epoch: - Shuffle the indices $1, \dots, n$ (random permutation). - For each i in shuffled order: $\theta \leftarrow \theta - \eta (\hat{y}^{(i)} - y^{(i)}) x^{(i)}$. 3. Repeat until convergence.

Why shuffle? Otherwise consecutive updates use correlated samples — e.g., data sorted by date would push θ in one direction for a long stretch.

Mini-batch Gradient Descent — the practical default

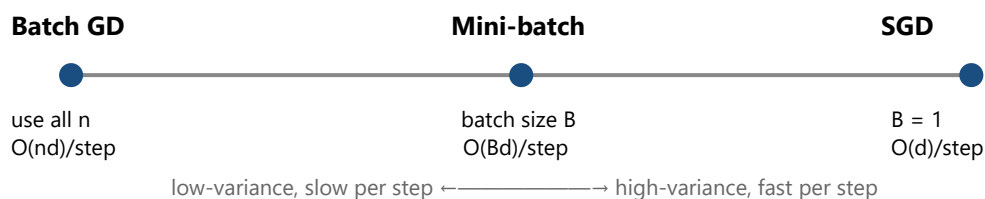
Take B samples at random (a "mini-batch" $\mathcal{B}$, $1 \leq B \leq n$), average their gradients:

$$\nabla_{\theta} L_{\mathcal{B}}(\theta) = \frac{1}{B} \sum_{i \in \mathcal{B}} (\hat{y}^{(i)} - y^{(i)}) x^{(i)} = \frac{1}{B} X_{\mathcal{B}}^{\top} (X_{\mathcal{B}} \theta - y_{\mathcal{B}}).$$

$$\theta \leftarrow \theta - \frac{\eta}{B} X_{\mathcal{B}}^{\top} (X_{\mathcal{B}} \theta - y_{\mathcal{B}}).$$

Cost per step: $O(Bd)$.

The spectrum



Trade-off: larger $B \rightarrow$ less noisy gradient (closer to true ∇L) but slower per step. Mini-batch with $B \in [32, 512]$ is the modern practical sweet spot.

Part B — Logistic Regression (L16)

The problem

Classification: given $x \in \mathbb{R}^d$, predict $y \in \{0, 1\}$. Examples: spam (1) vs not (0); chronic high blood pressure (1) vs not (0); fraud detection.

We want a hypothesis $h_\theta(x)$ that gives us $\mathbb{P}(y = 1 \mid x; \theta)$, then threshold at $\frac{1}{2}$.

Two failed candidates $\rightarrow$ the sigmoid

We need $h_\theta(x) \in [0, 1]$.

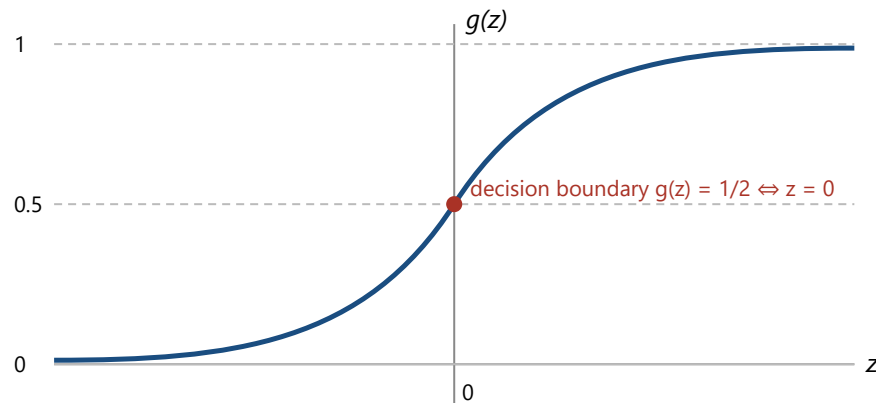
- **Candidate 1:** $h_\theta(x) = \theta^\top x$. Violates $h \in [0, 1]$ (can be any real number).
- **Candidate 2:** Move to odds. odds = $\frac{h}{1-h} \in (0, \infty)$. Set odds = $\theta^\top x$. Still wrong: $\theta^\top x$ can be negative, odds can't.
- **Candidate 3 (logistic regression): log-odds = $\theta^\top x$.**

$$\log \frac{h_\theta(x)}{1 - h_\theta(x)} = \theta^\top x.$$

Solving for h_θ gives the **sigmoid**:

$$h_\theta(x) = \frac{1}{1 + e^{-\theta^\top x}} = g(\theta^\top x), \quad g(z) = \frac{1}{1 + e^{-z}}.$$

Sigmoid — picture and key properties



Two identities that show up later:

$$g'(z) = g(z)(1 - g(z)), \quad 1 - g(z) = g(-z).$$

Decision rule

Predict $\hat{y} = 1$ if $h_{\theta}(x) > \frac{1}{2}$, else $\hat{y} = 0$. Since $g(0) = \frac{1}{2}$ and g is monotone, this is equivalent to:

$$\hat{y} = 1 \iff \theta^{\top} x > 0.$$

Geometric reading. The decision boundary $\{x : \theta^{\top} x = 0\}$ is a **hyperplane** through the origin. Logistic regression is a **linear classifier** — the boundary in feature space is linear, even though the probability curve is sigmoid.

How do we learn θ ? — Maximum likelihood

Three modelling assumptions: 1. Samples are i.i.d. 2. $\mathbb{P}(y = 1 \mid x; \theta) = h_{\theta}(x)$. 3. $y \mid x; \theta \sim \text{Bernoulli}(h_{\theta}(x))$, so $\mathbb{P}(y^{(i)} \mid x^{(i)}; \theta) = h^{y^{(i)}}(1 - h)^{1-y^{(i)}}$.

Likelihood:

$$L(\theta) = \prod_{i=1}^N h_{\theta}(x^{(i)})^{y^{(i)}} (1 - h_{\theta}(x^{(i)}))^{1-y^{(i)}}.$$

Maximising products is numerically nasty (underflow). Take the log; maximising $\log L$ is equivalent to minimising $-\log L$, which is the **cross-entropy loss**:

$$J(\theta) = - \sum_{i=1}^N \left[y^{(i)} \log h_{\theta}(x^{(i)}) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)})) \right].$$

Two facts that make this the right loss: - $J(\theta)$ is **convex** in θ . (Proof: each term is convex in $\theta^{\top} x$ — softplus-like — and the affine $x \mapsto \theta^{\top} x$ preserves convexity. Sum of convex is convex.) - No closed form for $\arg \min J$ — but convex + smooth, so **gradient descent finds the global minimum**.

Gradient and the GD update

A miraculous cancellation: the gradient of cross-entropy has the **same form** as the linear-regression gradient:

$$\nabla_{\theta} J(\theta) = \sum_{i=1}^N (h_{\theta}(x^{(i)}) - y^{(i)}) x^{(i)} = X^{\top} (g(X\theta) - y).$$

$$\theta \leftarrow \theta - \frac{\alpha}{N} X^{\top} (g(X\theta) - y).$$

Use SGD or mini-batch variants for large N — same machinery as Part A.

Part C — Worked sample problems

Problem 1 — One step of SGD by hand

You have linear regression with 3 samples:

$$x^{(1)} = (1, 2), y^{(1)} = 5; \quad x^{(2)} = (1, 0), y^{(2)} = 1; \quad x^{(3)} = (0, 1), y^{(3)} = 2.$$

Current $\theta = (1, 1)$, learning rate $\eta = 0.1$. Sample $i = 2$ is drawn. Compute the SGD update.

Solution. $\hat{y}^{(2)} = \theta^{\top} x^{(2)} = 1 \cdot 1 + 1 \cdot 0 = 1$. Residual $\hat{y}^{(2)} - y^{(2)} = 1 - 1 = 0$. So $\nabla L^{(2)} = 0 \cdot (1, 0) = (0, 0)$, and $\theta \leftarrow (1, 1) - 0.1 \cdot (0, 0) = (1, 1)$. **No update** this round — sample 2 already fits perfectly under current parameters.

Problem 2 — Decision boundary of logistic regression

After training, $\theta = (-3, 2, 1)$ for a model with intercept (so $x = (1, x_1, x_2)$). What's the decision boundary in the (x_1, x_2) plane?

Solution. Predict $\hat{y} = 1$ when $\theta^{\top} x > 0$, i.e., $-3 + 2x_1 + x_2 > 0 \iff x_2 > 3 - 2x_1$. The boundary is the line $x_2 = 3 - 2x_1$. Everything above the line is class 1.

Problem 3 — Sigmoid derivative identity

Show $g'(z) = g(z)(1 - g(z))$ where $g(z) = (1 + e^{-z})^{-1}$.

Solution. Quotient/chain rule:

$$g'(z) = -(1 + e^{-z})^{-2} \cdot (-e^{-z}) = \frac{e^{-z}}{(1 + e^{-z})^2} = \frac{1}{1 + e^{-z}} \cdot \frac{e^{-z}}{1 + e^{-z}} = g(z) \cdot (1 - g(z)).$$

Problem 4 — Cross-entropy gradient (key derivation)

Show that for one sample, $\nabla_{\theta} J^{(i)}(\theta) = (h_{\theta}(x^{(i)}) - y^{(i)}) x^{(i)}$.

Solution. Drop the i for brevity. With $h = g(\theta^{\top} x)$ and $z = \theta^{\top} x$:

$$J = -[y \log h + (1 - y) \log(1 - h)].$$

$$\frac{\partial J}{\partial \theta_j} = -\left[\frac{y}{h} - \frac{1-y}{1-h}\right] \cdot \frac{\partial h}{\partial \theta_j} = -\left[\frac{y(1-h) - (1-y)h}{h(1-h)}\right] \cdot g'(z) x_j = -\left[\frac{y-h}{h(1-h)}\right] \cdot h(1-h) x_j = (h-y) x_j.$$

Done.

Problem 5 — Why SGD is unbiased

Show that if i is drawn uniformly from $\{1, \dots, n\}$, then $\mathbb{E}_i[\nabla_\theta L^{(i)}(\theta)] = \nabla_\theta L(\theta)$ for $L = \frac{1}{n} \sum_i L^{(i)}$.

Solution. $\mathbb{E}_i[\nabla L^{(i)}] = \sum_i \frac{1}{n} \nabla L^{(i)} = \nabla(\frac{1}{n} \sum_i L^{(i)}) = \nabla L$. (Linearity of gradient + expectation.)

That's why "on average, SGD moves in the right direction."

Problem 6 — Compare cost per epoch

You have $n = 10^6$, $d = 100$. Batch GD does one update using all data; SGD does one update per sample. How many gradient evaluations per *epoch* (= one full pass over the data) does each do, and what's the FLOP comparison?

Solution. - Batch: 1 step per epoch, $O(nd) = 10^8$ ops. - SGD: $n = 10^6$ steps per epoch, $O(d) = 100$ ops each, total 10^8 ops.

Same total work per epoch! SGD doesn't save work — it saves *time to first progress*. After one SGD step you've already moved θ ; after one batch step you've only moved once and burned the same total compute.

Problem 7 — Logistic regression on a tiny dataset

Single feature, θ_0 = bias, θ_1 = slope. Two training points: $(x, y) = (1, 1)$ and $(-1, 0)$. Show that $\theta_0 = 0, \theta_1 \rightarrow +\infty$ is the MLE (no finite optimum) — the data is **linearly separable**.

Solution. $P(y = 1 | x = 1) = g(\theta_1)$. $P(y = 0 | x = -1) = 1 - g(-\theta_1) = g(\theta_1)$. Likelihood $L = g(\theta_1)^2$. Maximised as $\theta_1 \rightarrow \infty$ (sigmoid $\rightarrow 1$). The decision boundary $\theta_1 x = 0$ classifies the data perfectly for *any* $\theta_1 > 0$, but the likelihood keeps increasing. **Take-away:** on linearly separable data, MLE for logistic regression diverges. Regularisation (L2) fixes this.

Part D — Cheat-sheet take-aways

- $\nabla L^{(i)} = (\hat{y}^{(i)} - y^{(i)}) x^{(i)}$ for squared loss, $(h - y) x$ for cross-entropy — **same form**.
- GD: $\theta \leftarrow \theta - \eta \nabla L$. SGD: one sample, $O(d)$. Mini-batch: B samples, $O(Bd)$.
- SGD is unbiased ($\mathbb{E}[\nabla L^{(i)}] = \nabla L$); decaying $\eta_t = 1/\sqrt{t}$ controls variance.
- Logistic regression: $h_\theta(x) = \sigma(\theta^\top x)$; linear decision boundary at $\theta^\top x = 0$.
- Loss = **cross-entropy** = $-\sum [y \log h + (1 - y) \log(1 - h)]$; convex, no closed form, train with GD.
- Linearly separable data $\Rightarrow$ unregularised MLE diverges; add L2.

02_GenerativeClassifiers_GDA_NaiveBayes_Explainer

Post-Mid-Sem 2: Generative Classifiers — GDA + Naive Bayes

Lectures 17 (GDA) and 18 (Naive Bayes). Both are **generative** models — they model how the data is generated (the class first, then the features given the class). Compare with logistic regression, which directly models $P(y | x)$ (discriminative).

The two philosophies of classification

	Discriminative	Generative
Models	$P(y x)$ directly	$P(x y)$ and $P(y)$, then combines via Bayes
Example	Logistic regression	GDA, Naive Bayes
Question it answers	"Given the features, which class?"	"What does a typical sample from each class look like?"
To predict	Output $h_\theta(x)$ and threshold	Compute $P(y x) \propto P(x y)P(y)$ for each y , take argmax

Bayes' rule is what lets a generative model do classification:

$$P(y | x) = \frac{P(x | y) P(y)}{P(x)}.$$

Because we want $\arg \max_y P(y | x)$ and $P(x)$ doesn't depend on y , we just need to maximise $P(x | y)P(y)$.

Discriminative

Learn: $P(y | x)$
e.g., logistic regression

- + usually higher accuracy
- + no model assumption on x
- cannot generate samples
- needs more data

Generative

Learn: $P(x | y)$, $P(y)$
e.g., GDA, Naive Bayes

- + closed-form MLE
- + works with little data
- + can sample new x
- strong model assumptions

Part A — Gaussian Discriminant Analysis (GDA)

Prerequisite: the Gaussian

Univariate: $X \sim \mathcal{N}(\mu, \sigma^2)$,

$$f_X(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right).$$

Multivariate (d -dimensional): $X \sim \mathcal{N}(\boldsymbol{\mu}, \Sigma)$ with $\boldsymbol{\mu} \in \mathbb{R}^d$, $\Sigma \in \mathbb{R}^{d \times d}$ symmetric PSD,

$$f_X(x) = \frac{1}{(2\pi)^{d/2} |\Sigma|^{1/2}} \exp\left(-\frac{1}{2}(x - \boldsymbol{\mu})^\top \Sigma^{-1}(x - \boldsymbol{\mu})\right).$$

Reading the covariance (for $d = 2$):

$$\Sigma = \begin{pmatrix} \sigma_1^2 & \sigma_{12} \\ \sigma_{12} & \sigma_2^2 \end{pmatrix}, \quad \sigma_{12} = \text{Cov}(X_1, X_2) = \mathbb{E}[(X_1 - \mu_1)(X_2 - \mu_2)].$$

- $\sigma_{12} > 0$: higher $x_1 \Rightarrow$ higher x_2 likely (positive correlation, ellipse tilts up-right).
- $\sigma_{12} < 0$: higher $x_1 \Rightarrow$ lower x_2 (tilts up-left).
- $\sigma_{12} = 0$: axis-aligned ellipse.

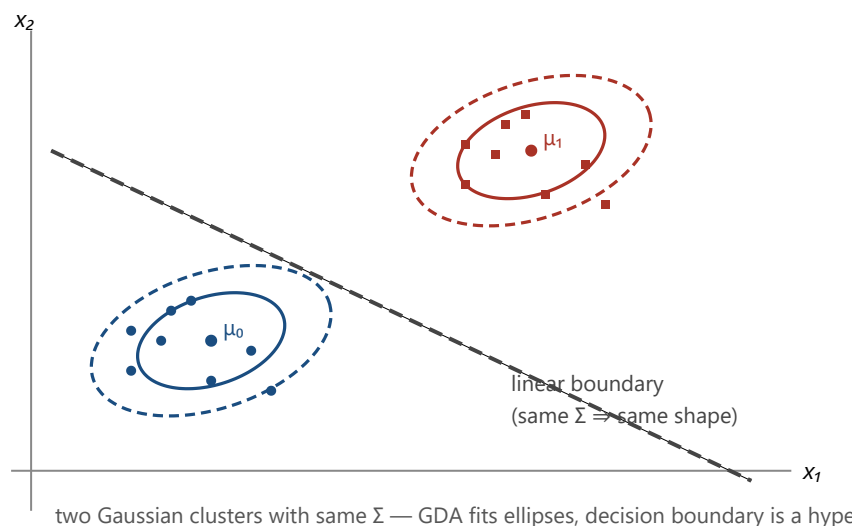
GDA modelling assumptions (binary case)

1. **Class prior:** $y \sim \text{Bernoulli}(\phi)$, so $P(y = 1) = \phi$, $P(y = 0) = 1 - \phi$.
2. **Class-conditional features:**

$$x \mid y = 0 \sim \mathcal{N}(\boldsymbol{\mu}_0, \Sigma), \quad x \mid y = 1 \sim \mathcal{N}(\boldsymbol{\mu}_1, \Sigma).$$

Crucial detail: the covariance Σ is **shared** across both classes. The classes have different means, same shape.

Parameters to learn: ϕ (scalar), $\boldsymbol{\mu}_0, \boldsymbol{\mu}_1$ (d each), Σ ($O(d^2)$). Total: $O(d^2)$ parameters (dominated by Σ).



Parameter MLE (closed form, $O(Nd^2)$)

Maximising $L(\phi, \mu_0, \mu_1, \Sigma) = \prod_i P(x^{(i)} | y^{(i)}) P(y^{(i)})$ gives:

$$\phi^* = \frac{1}{N} \sum_i \mathbb{1}\{y^{(i)} = 1\}, \quad \mu_k^* = \frac{\sum_{i: y^{(i)} = k} x^{(i)}}{\sum_i \mathbb{1}\{y^{(i)} = k\}},$$

$$\Sigma^* = \frac{1}{N} \sum_{i=1}^N (x^{(i)} - \mu_{y^{(i)}}^*)(x^{(i)} - \mu_{y^{(i)}}^*)^\top.$$

In words: ϕ = fraction class 1; μ_k = sample mean of class k ; Σ = pooled sample covariance using each class's own mean.

Prediction and the linear decision boundary

To classify a new x :

$$\hat{y} = \arg \max_y P(y | x) = \arg \max_y P(x | y) P(y).$$

Predict $\hat{y} = 1$ iff $P(x | y = 1)P(y = 1) \geq P(x | y = 0)P(y = 0)$. Plug in the Gaussian densities, take logs, and **everything quadratic in x cancels** (because Σ is the same on both sides). What's left is linear in x :

$$(\mu_1 - \mu_0)^\top \Sigma^{-1} x + \underbrace{\left(-\frac{1}{2} \mu_1^\top \Sigma^{-1} \mu_1 + \frac{1}{2} \mu_0^\top \Sigma^{-1} \mu_0 + \log \frac{\phi}{1-\phi}\right)}_{\theta_0} = 0.$$

Writing $\theta = \Sigma^{-1}(\mu_1 - \mu_0)$, this is $\theta^\top x + \theta_0 = 0$ — a **hyperplane**, just like logistic regression. (In fact one can prove $P(y | x)$ under GDA's assumptions is exactly the sigmoid — GDA *implies* logistic regression, but the converse is not true.)

What if the two classes have different Σ_0, Σ_1 ? Then the quadratic terms don't cancel and the boundary becomes a **quadratic** curve (a conic). That's called QDA.

Part B — Naive Bayes

Why we need Naive Bayes — the spam-filter parameter count

Spam classification with $d = 50,000$ binary word features ($x_j = 1$ if word j appears in the email):

Model	Parameters
Class prior ϕ	1
Mean per class	50,000 each
Covariance Σ (shared)	$50,000^2 / 2 \approx 1.25 \cdot 10^9$

The covariance matrix alone has **a billion** parameters. We don't have a billion labelled emails. GDA is **intractable** here.

The Naive Bayes assumption — conditional independence

We still want to use Bayes' rule. The exact chain-rule expansion is

$$P(x_1, x_2, \dots, x_d | y) = P(x_1 | y) P(x_2 | y, x_1) P(x_3 | y, x_1, x_2) \dots$$

Each conditional gets more complex. Naive Bayes makes a brutal simplification — **given the class, features are independent**:

$$P(x_1, \dots, x_d | y) = \prod_{j=1}^d P(x_j | y).$$

This drops the conditioning on other features in every term. For spam: knowing the email is spam, NB pretends seeing "free" tells you nothing extra about seeing "offer". **Wrong**, but the assumption keeps the model usable: we only need one $P(x_j | y)$ per feature per class.

A famous statistician's aphorism applies here: *"All models are wrong, but some are useful."*

NB modelling assumptions (binary features, K classes)

1. $P(y = k) = \phi_k$ with $\sum_{k=0}^{K-1} \phi_k = 1$.
2. $P(x | y = k) = \prod_{j=1}^d P(x_j | y = k)$ (conditional independence).
3. $x_j | y = k \sim \text{Bernoulli}(\phi_{j|k})$, so $P(x_j = 1 | y = k) = \phi_{j|k}$.

Total parameters: $(d + 1)K$ — linear in d , not quadratic. For spam with $d = 50,000$, $K = 2$: about 10^5 . **Tractable**.

MLE for NB (closed form, $O(Nd)$)

The log-likelihood factors beautifully across class priors and per-feature, per-class Bernoulli parameters:

$$\phi_k^* = \frac{\text{count}(y^{(i)} = k)}{N}, \quad \phi_{j|k}^* = \frac{\text{count}(x_j^{(i)} = 1 \text{ AND } y^{(i)} = k)}{\text{count}(y^{(i)} = k)}.$$

Just **count fractions**. No iterative optimisation, no matrix inversion. Insanely simple to implement.

Prediction

$$\hat{y} = \arg \max_k \phi_k^* \prod_{j=1}^d (\phi_{j|k}^*)^{x_j} (1 - \phi_{j|k}^*)^{1-x_j}.$$

In practice take logs to avoid underflow:

$$\hat{y} = \arg \max_k \log \phi_k^* + \sum_{j=1}^d \left[x_j \log \phi_{j|k}^* + (1 - x_j) \log(1 - \phi_{j|k}^*) \right].$$

The zero-probability disaster, and Laplace smoothing

If a word j never appears with class k in training, $\phi_{j|k}^* = 0$. Then any test email containing that word multiplies the entire class- k posterior by 0 — even if every other word screams “class k ”. This is the **zero-frequency problem**.

Fix — Laplace (add-one) smoothing. Pretend you saw every (word, class) pair exactly once extra:

$$\hat{\phi}_{j|k} = \frac{\text{count}(x_j = 1, y = k) + 1}{\text{count}(y = k) + V}$$

where V is the number of possible values x_j can take ($V = 2$ for binary). Now $\hat{\phi}_{j|k} \in (0, 1)$ always — no more zeros.

Independence vs conditional independence — keep them straight

	Statement	When true
Marginal independence	$A \perp B$	$P(A \cap B) = P(A) P(B)$
Conditional independence	$A \perp B \mid C$	$P(A \cap B \mid C) = P(A \mid C) P(B \mid C)$

These do **not** imply each other. NB asserts the second, not the first. Spam words *are* correlated overall (“free” and “offer” co-occur); NB’s claim is just that *within* the spam class their occurrence is independent.

Part C — Worked sample problems

Problem 1 — GDA from a tiny dataset

Two features, 4 training points: - Class 0: (1, 1), (2, 2) - Class 1: (4, 5), (5, 4)

Compute the GDA MLE estimates $\phi, \mu_0, \mu_1, \Sigma$ and the predicted class for the new point $x = (3, 3)$.

Solution. $\phi^* = 2/4 = 0.5$. $\mu_0^* = ((1 + 2)/2, (1 + 2)/2) = (1.5, 1.5)$.

$\mu_1^* = ((4 + 5)/2, (5 + 4)/2) = (4.5, 4.5)$.

Centered deviations: - Class 0: $(1, 1) - (1.5, 1.5) = (-0.5, -0.5)$, $(2, 2) - (1.5, 1.5) = (0.5, 0.5)$ - Class 1: $(4, 5) - (4.5, 4.5) = (-0.5, 0.5)$, $(5, 4) - (4.5, 4.5) = (0.5, -0.5)$

Pool the outer products: - $(-0.5, -0.5)^\top (-0.5, -0.5) = \begin{pmatrix} 0.25 & 0.25 \\ 0.25 & 0.25 \end{pmatrix}$; same for $(0.5, 0.5)$. -

$(-0.5, 0.5)^\top (-0.5, 0.5) = \begin{pmatrix} 0.25 & -0.25 \\ -0.25 & 0.25 \end{pmatrix}$; same for $(0.5, -0.5)$.

Sum: $\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$. Divide by $N = 4$: $\Sigma^* = \frac{1}{4}I$.

For prediction at $(3, 3)$: distances squared (Mahalanobis with $\Sigma^* = I/4 \Rightarrow \Sigma^{*-1} = 4I$) to each mean: - to μ_0 : $4 \cdot ((1.5)^2 + (1.5)^2) = 4 \cdot 4.5 = 18$. - to μ_1 : $4 \cdot ((-1.5)^2 + (-1.5)^2) = 18$. Equal Mahalanobis distances. With $\phi = 0.5$ priors, the posteriors are equal — predict either class, or break ties arbitrarily. $(3, 3)$ is exactly on the decision boundary.

Problem 2 — NB MLE on a 4-row table

Predict spam ($y = 1$) vs not ($y = 0$) from two binary words "free" (x_1) and "win" (x_2). Training data:

email	x_1 (free)	x_2 (win)	y (spam)
1	1	1	1
2	1	0	1
3	0	0	0
4	0	1	0

Compute MLE without and with Laplace smoothing. Classify $(x_1 = 1, x_2 = 1)$.

Solution. $N = 4$, $\text{count}(y = 1) = 2$, $\text{count}(y = 0) = 2$. $\phi_1^* = 0.5$, $\phi_0^* = 0.5$.

Without smoothing: - $\phi_{1|1}^* = \text{count}(x_1 = 1, y = 1)/2 = 2/2 = 1$ - $\phi_{2|1}^* = 1/2$ - $\phi_{1|0}^* = 0/2 = 0$ - $\phi_{2|0}^* = 1/2$

With Laplace smoothing ($V = 2$, add 1 to numerator and 2 to denominator): -

$\hat{\phi}_{1|1} = (2 + 1)/(2 + 2) = 3/4$ - $\hat{\phi}_{2|1} = (1 + 1)/(2 + 2) = 1/2$ - $\hat{\phi}_{1|0} = (0 + 1)/(2 + 2) = 1/4$ - $\hat{\phi}_{2|0} = (1 + 1)/(2 + 2) = 1/2$

Classify $(1, 1)$ with smoothed estimates: - Class 1 score: $0.5 \cdot 3/4 \cdot 1/2 = 3/16 = 0.1875$ - Class 0 score: $0.5 \cdot 1/4 \cdot 1/2 = 1/16 = 0.0625$

Predict $\hat{y} = 1$. (Without smoothing, class-0 would have gotten a deceptive 0 even though "win" appears in class 0 too.)

Problem 3 — Why GDA's boundary is linear when Σ is shared

Sketch the proof.

Solution. Predict class 1 iff $\log P(x | 1) + \log P(1) \geq \log P(x | 0) + \log P(0)$. Multiply out the Gaussian log-densities:

$$-\frac{1}{2}(x - \mu_1)^\top \Sigma^{-1}(x - \mu_1) + \log \phi \geq -\frac{1}{2}(x - \mu_0)^\top \Sigma^{-1}(x - \mu_0) + \log(1 - \phi).$$

The $-\frac{1}{2}\log |\Sigma|$ terms are equal on both sides (same Σ) and cancel. Expand the quadratic forms: $-\frac{1}{2}x^\top \Sigma^{-1}x$ also appears on both sides and cancels. What's left is linear in x : $\theta^\top x + \theta_0 \geq 0$ with $\theta = \Sigma^{-1}(\mu_1 - \mu_0)$. **Take-away:** shared $\Sigma \Rightarrow$ linear boundary; different $\Sigma_0, \Sigma_1 \Rightarrow$ quadratic (QDA).

Problem 4 — GDA vs Logistic Regression

GDA implies $P(y | x)$ is a sigmoid in $\theta^\top x$. So GDA and logistic regression both produce linear decision boundaries. Why might you prefer one over the other?

Solution. GDA makes a *stronger* assumption (the Gaussian shape). If that assumption holds, GDA needs fewer training samples to estimate the boundary well (it's making more efficient use of the data through the model). If the assumption is wrong, logistic regression is more robust because it commits only to the boundary shape, not to the class-conditional densities. Rule of thumb: **GDA wins when the Gaussian assumption is approximately true; logistic regression wins when it's not.**

Problem 5 — Bayes' rule classification by hand

You have $P(y = 1) = 0.3$. For a feature $x \in \{a, b\}$: $P(x = a | y = 1) = 0.8$, $P(x = a | y = 0) = 0.2$. Classify a sample with $x = a$.

Solution. - $P(x = a | y = 1)P(y = 1) = 0.8 \cdot 0.3 = 0.24$. - $P(x = a | y = 0)P(y = 0) = 0.2 \cdot 0.7 = 0.14$. Predict $\hat{y} = 1$. (For the actual posterior: $P(x = a) = 0.24 + 0.14 = 0.38$, so $P(y = 1 | x = a) = 0.24/0.38 \approx 0.63$.)

Problem 6 — Naive Bayes for continuous features

How would you adapt NB if features were continuous rather than binary?

Solution. Replace the per-feature Bernoulli assumption with a per-feature Gaussian: $x_j | y = k \sim \mathcal{N}(\mu_{j|k}, \sigma_{j|k}^2)$. Now MLE estimates the per-feature, per-class mean and variance. This is called **Gaussian Naive Bayes**. It's equivalent to GDA where Σ is forced to be *diagonal* (and even allowed to differ per class — a strictly weaker model than full GDA but still tractable when full GDA isn't).

Problem 7 — Conditional vs marginal independence

Are spam words "free" and "offer" marginally independent? Conditionally independent given the class label? Which assumption does NB make?

Solution. They are clearly *not* marginally independent — both occur more often in spam than in ham, so observing one raises the probability of the other across the dataset as a whole. Conditional independence given y is the **NB assumption**: *within* the spam class, the occurrence of "free" doesn't tell you anything additional about "offer". In reality even this is false (spam emails often contain promotional phrases together), but NB tolerates the violation and still classifies well.

Part D — Cheat-sheet take-aways

- **Bayes' rule for classification:** $P(y | x) \propto P(x | y) P(y)$.
- **GDA:** $x | y = k \sim \mathcal{N}(\mu_k, \Sigma)$ with **shared** Σ . Closed-form MLE: class mean per class, pooled covariance. Decision boundary is **linear** in x . $O(d^2)$ parameters.
- **Naive Bayes:** assumes $P(x | y) = \prod_j P(x_j | y)$ (conditional independence). $(d + 1)K$ parameters. MLE = count fractions, $O(Nd)$.

- **Laplace smoothing:** $\hat{\phi}_{j|k} = (c + 1)/(n_k + V)$. Avoids zero-probability multiplications.
- GDA $\Rightarrow$ logistic regression form for $P(y | x)$. Use GDA when the Gaussian assumption fits; use LR otherwise.
- "All models are wrong, but some are useful." NB violates conditional independence in practice and still works in many text-classification settings.

03_DecisionTrees_Ensembles_Explainer

Post-Mid-Sem 3: Decision Trees + Ensembles

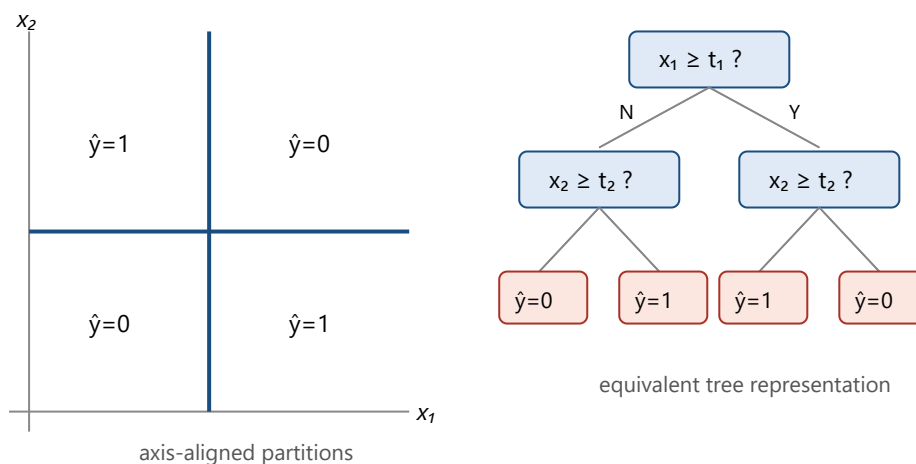
Lectures 19, 20, 21. Trees handle non-linear boundaries and feature interactions that logistic regression / NB / GDA cannot. They overfit easily, so we either prune them or **average many of them** (bagging, random forests).

Part A — Why decision trees?

Every classifier we've seen so far produces either a **hyperplane** ($\theta^T x = 0$ — LR, GDA) or a **piecewise-constant** rule of a special form (NB). None can naturally express rules like

" $\hat{y} = 1$ if $x_1 < t_1$ AND $x_2 > t_2$, OR if $x_1 > t_1$ AND $x_2 < t_2$ "

— a XOR-style region. Worse, none of them naturally express **feature interactions** like "income matters only when credit is bad." A decision tree does both directly.



Definition (decision tree)

1. **Binary tree.**
2. **Each internal node** stores a *split question* of the form " $x_j \geq s?$ " (one feature, one split value).
3. **Each leaf node** stores a predicted class (classification) or value (regression).

Important: the number of nodes and internal-node parameters is **not fixed in advance** — they're determined during training. So unlike LR (which has $d + 1$ params), DT has variable structure.

Part B — Building a tree for classification

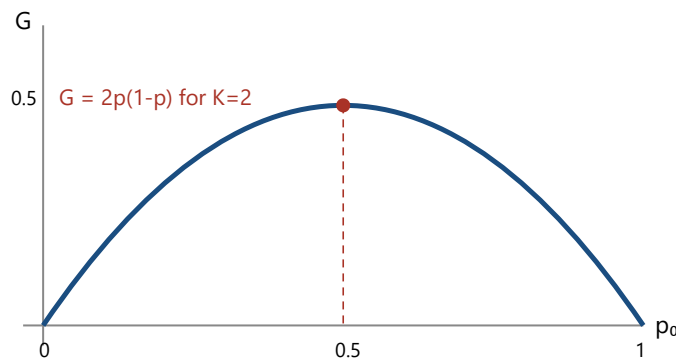
Measuring the goodness of a split — Gini impurity

For a node holding the sample set S with K classes, let $p_k = |S_k|/|S|$ be the fraction of class k in S . The **Gini impurity** is

$$G(S) = \sum_{k=0}^{K-1} p_k(1 - p_k) = 1 - \sum_{k=0}^{K-1} p_k^2.$$

Interpretation: the probability you'd get wrong if you randomly labelled a sample in S according to the class proportions. Pure node ($p_k = 1$ for some k) $\Rightarrow G = 0$. Most uncertain ($p_k = 1/K$ for all k) $\Rightarrow G = 1 - 1/K$ (max).

Binary case ($K = 2$, $p_0 = p$, $p_1 = 1 - p$): $G(p) = 2p(1 - p)$. Max $G = 0.5$ at $p = 0.5$.



Weighted Gini impurity (how a split is scored)

A split partitions S into S^- , S^+ . The **weighted** Gini impurity of the split is

$$\text{wGI}(S^-, S^+) = \frac{|S^-|}{|S|} G(S^-) + \frac{|S^+|}{|S|} G(S^+).$$

We pick the split with the **lowest** weighted Gini — most purity-creating per fraction of samples.

Candidate splits — only finitely many

For a continuous feature x_j , there are infinitely many threshold values. But within a given training set we only need to try $m - 1$ candidates, where m = number of distinct values x_j takes. Sort those values $v_1 < v_2 < \dots < v_m$. The candidates are midpoints

$$s_i = \frac{v_i + v_{i+1}}{2}.$$

Any threshold within the gap (v_i, v_{i+1}) produces the same partition of samples.

The greedy recursive algorithm

```

BuildTree(I):           # I = indices of training samples at this node
  1. If I is pure OR |I| < k OR depth > D:  # stopping rules
    return Leaf(majority class of I)
  2. For each feature j:
    For each candidate split value s:
      Partition I into I- = {i ∈ I : xj(i) < s}, I+ = {... ≥ s}
      Compute wGI(I-, I+)
    Pick (j*, s*) with minimum wGI
  3. Recurse: left = BuildTree(I-), right = BuildTree(I+)
  4. Return InternalNode(split: x{j*} ≥ s*, left, right)

```

Complexity per recursive call: $O(N d m)$ in the naive form (every feature × every candidate split × scanning samples). Greedy: each split is chosen to be best *locally*, no backtracking — globally optimal trees are NP-hard.

Part C — Overfitting and pruning

A fully grown tree can drive training error to zero by making each leaf hold a single sample. That's textbook overfitting. Two remedies:

Pre-pruning (early stopping)

Add stopping conditions to step 1 of the algorithm: - **Option 1** (size): stop if $|I| < k$ for some chosen k , or if I is pure. - **Option 2** (depth): stop if tree depth exceeds D . - **Option 3** (impurity gain): stop if the *improvement* in weighted Gini from the best split is below some threshold.

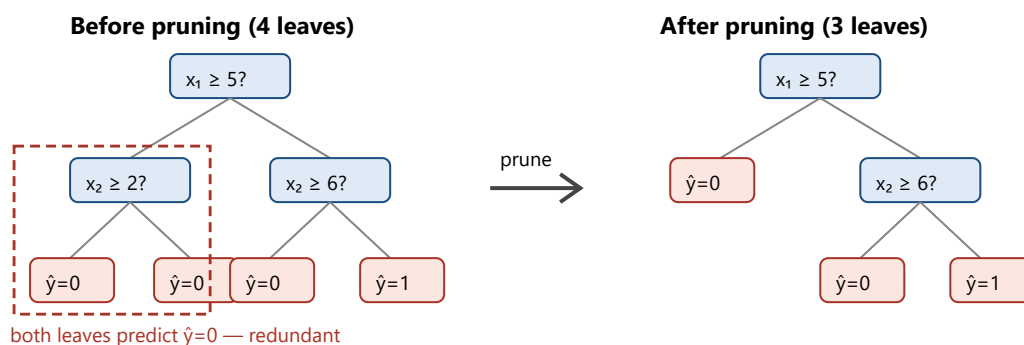
Post-pruning (cost-complexity pruning)

1. Build a **large** (overfit) tree.
2. Define a cost-complexity functional with hyperparameter $\alpha \geq 0$:

$$C_\alpha(T) = \text{train_error}(T) + \alpha \cdot |\text{leaves}(T)|.$$

3. Iteratively collapse leaves to find the subtree minimising C_α .
4. **Choose α by cross-validation** (CV picks the right model complexity).

Why post-pruning is preferable: early stopping is “look-ahead blind” — a split that *looks* bad now might enable a great split below it. Post-pruning sees the whole tree and trims globally.



Part D — Regression trees

Same tree structure, but each leaf stores a **real number** (predicted $\hat{y} \in \mathbb{R}$). Two changes:

1. **Leaf prediction:** mean of y values in that leaf,

$$\bar{y}_I = \frac{1}{|I|} \sum_{i \in I} y^{(i)}.$$

2. **Split criterion:** replace Gini with **sum of squared errors (SSE)**:

$$\text{SSE}(I) = \sum_{i \in I} (y^{(i)} - \bar{y}_I)^2.$$

Pick the split that minimises weighted SSE: $\frac{|I^-|}{|I|} \text{SSE}(I^-) + \frac{|I^+|}{|I|} \text{SSE}(I^+)$.

That's it — everything else (greedy recursion, pruning) is identical.

Reading the names: SSE small $\Rightarrow y$ values in the leaf are clustered tightly $\Rightarrow$ low intra-leaf variance $\Rightarrow$ good predictions. Same role as low Gini in classification.

Part E — Ensembling: why average many models?

Trees are **high-variance**: small changes in training data $\Rightarrow$ a different tree. Ensembling reduces variance by averaging.

Toy intuition

3 independent binary classifiers, each with error probability ϵ . Predict by **majority vote**. The ensemble errs iff at least 2 of 3 err:

$$P(\text{ensemble error}) = \binom{3}{2} \epsilon^2 (1 - \epsilon) + \epsilon^3 = 3\epsilon^2 (1 - \epsilon) + \epsilon^3.$$

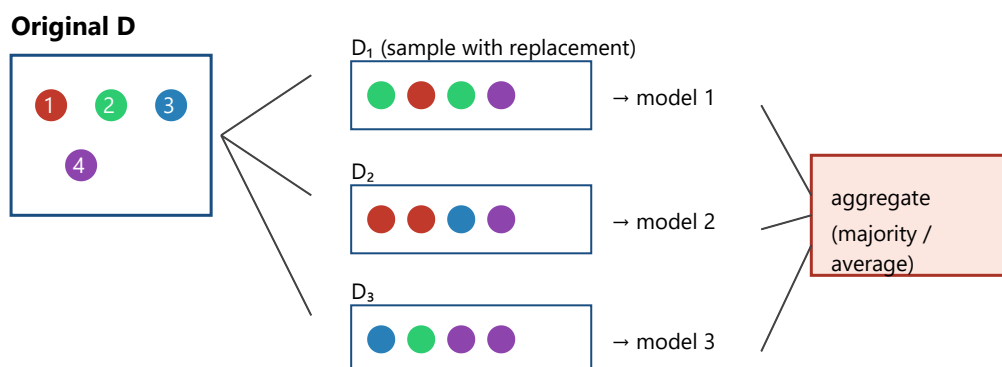
Compare to a single classifier's error ϵ . If $\epsilon < 1/2$, you can show $3\epsilon^2 (1 - \epsilon) + \epsilon^3 < \epsilon$. **The ensemble**

does strictly better than its members — provided members are at least slightly better than random *and* their errors are independent.

Bootstrapping — fake “multiple datasets”

We don't have multiple independent training sets. Trick: **sample with replacement** from the one we have.

Bootstrap dataset = N samples drawn with replacement from the N -sample training set. Some originals appear multiple times, others not at all (~37% of originals appear in no draw, on average).



Bagging (Bootstrap AGGREGatING)

```
For b = 1, 2, ..., B:
  a) Bootstrap dataset D_b from D
  b) Train your model on D_b
Predict:
  Classification: majority vote across B models
  Regression: average across B models
```

Bagging reduces variance dramatically while leaving bias roughly unchanged — perfect for high-variance learners like deep trees.

Random Forest = Bagging + Decision Trees + extra randomness

Plain bagging trees still have **correlated errors** (they all consider the same features at each split, so they tend to pick similar top splits). Random forest decorrelates them:

```
For b = 1, ..., B:
  a) Bootstrap D_b
  b) Build a decision tree on D_b, but for EACH split:
      - Pick m out of d features uniformly at random (without replacement)
      - Search the best split only over those m features
Predict by aggregation (majority/average).
```

Typical choices: $m = \sqrt{d}$ for classification, $m = d/3$ for regression. Picking $m \ll d$ forces different trees to use different features and decorrelates them. **Random forests are usually one of the**

strongest off-the-shelf classifiers and require almost no tuning.

Part F — Worked sample problems

Problem 1 — Gini impurity by hand

A node has 6 samples: 4 of class 0, 2 of class 1. Compute Gini impurity. Then evaluate two candidate splits and pick the better one.

Split A: {class 0, class 0, class 1} vs {class 0, class 0, class 1}. Split B: {class 0, class 0, class 0, class 0} vs {class 1, class 1}.

Solution. Parent: $p_0 = 2/3, p_1 = 1/3, G = 1 - (4/9 + 1/9) = 4/9 \approx 0.444$.

Split A — each child: $p_0 = 2/3, p_1 = 1/3, G = 4/9$. Weighted: $\frac{3}{6} \cdot \frac{4}{9} + \frac{3}{6} \cdot \frac{4}{9} = 4/9 \approx 0.444$. **No improvement** — this split tells us nothing.

Split B — child 1: pure ($G = 0$). Child 2: pure ($G = 0$). Weighted: 0. **Perfect split** — both children fully separate the classes.

Pick **split B**. Gini reduction = $4/9 - 0 = 4/9$.

Problem 2 — Pick candidate splits for a feature

A feature x_1 takes values {1.2, 3.5, 4.0, 7.1} across training samples. What are the candidate split values?

Solution. Midpoints of consecutive sorted values: $(1.2 + 3.5)/2 = 2.35, (3.5 + 4.0)/2 = 3.75, (4.0 + 7.1)/2 = 5.55$. Three candidate splits.

Problem 3 — Predict from a tree

Tree: root asks " $x_1 \geq 5$ ". No branch leads to a node asking " $x_2 \geq 2$ ", whose No branch is $\hat{y} = 0$, Yes branch is $\hat{y} = 1$. The Yes branch of the root is $\hat{y} = 1$. Predict for $x = (3, 4)$.

Solution. Root: $x_1 = 3 \not\geq 5$, go No. Then $x_2 = 4 \geq 2$, go Yes. Predict $\hat{y} = 1$.

Problem 4 — Regression tree split

Samples (one feature, target): (1, 2), (2, 3), (3, 8), (4, 9). Score the split " $x \geq 2.5$ ".

Solution. $\bar{y} = (2 + 3 + 8 + 9)/4 = 5.5$. Parent SSE
 $= (2 - 5.5)^2 + (3 - 5.5)^2 + (8 - 5.5)^2 + (9 - 5.5)^2 = 12.25 + 6.25 + 6.25 + 12.25 = 37$.

Left ($x < 2.5$): {2, 3}, $\bar{y} = 2.5$. SSE = $(2 - 2.5)^2 + (3 - 2.5)^2 = 0.25 + 0.25 = 0.5$. Right ($x \geq 2.5$): {8, 9}, $\bar{y} = 8.5$. SSE = 0.5.

Weighted: $\frac{2}{4} \cdot 0.5 + \frac{2}{4} \cdot 0.5 = 0.5$. SSE reduction = $37 - 0.5 = 36.5$. Huge improvement.

Problem 5 — Cost-complexity pruning

Tree T has training error 0.1 and 6 leaves. Subtree T' has training error 0.15 and 3 leaves. For what α would post-pruning prefer T' ?

Solution. Prefer T' iff $C_\alpha(T') \leq C_\alpha(T)$:

$0.15 + 3\alpha \leq 0.1 + 6\alpha \Rightarrow 0.05 \leq 3\alpha \Rightarrow \alpha \geq 1/60 \approx 0.0167$. So for any $\alpha \geq 1/60$, the smaller tree wins.

Problem 6 — Why bagging works

Three independent classifiers each have error $\epsilon = 0.3$. Compute the majority-vote ensemble error.

Solution. $P(\text{err}) = \binom{3}{2}(0.3)^2(0.7) + (0.3)^3 = 3 \cdot 0.09 \cdot 0.7 + 0.027 = 0.189 + 0.027 = 0.216$.

Reduced from 0.30 to 0.216. The bigger B , the closer the ensemble error gets to 0 (assuming independence, which is the assumption that breaks in practice — hence the decorrelation tricks in Random Forest).

Problem 7 — Choosing m in Random Forest

You have $d = 100$ features and a classification task. What value of m would you use? What if it were regression?

Solution. Rule of thumb: $m = \sqrt{d} = 10$ for classification; $m = d/3 \approx 33$ for regression. These are defaults — you'd cross-validate around them for best results.

Problem 8 — Why “high variance” is the *right* word for trees

A single fully grown tree on dataset D vs the same algorithm on D' (very similar) gives wildly different splits at the root. Why?

Solution. Splits are chosen *greedily* using exact threshold comparisons on training data. A small change in the data can flip which feature wins the root split (e.g., two features tied at the root); after that, the rest of the tree is completely different (because the partition changed). Small perturbation in data $\rightarrow$ large change in model = **high variance**. Bagging fights this by averaging many such randomly perturbed trees, so the wild fluctuations cancel.

Part G — Cheat-sheet take-aways

- **Decision tree:** binary, internal nodes ask “ $x_j \geq s$?”, leaves predict class/value.
- **Split criterion:**
 - Classification: minimise **weighted Gini** $\frac{|I^-|}{|I|}G(I^-) + \frac{|I^+|}{|I|}G(I^+)$ with $G(S) = 1 - \sum p_k^2$.
 - Regression: minimise **weighted SSE** with $\text{SSE}(I) = \sum_{i \in I} (y^{(i)} - \bar{y}_I)^2$.
- **Build algorithm:** greedy recursive. Stop on purity / size / depth.
- **Pruning:** pre-prune (early stopping) or post-prune via $C_\alpha(T) = \text{train_err} + \alpha \cdot |\text{leaves}|$; tune α by CV.
- **Bagging:** B models trained on bootstrap samples; aggregate by majority/average. Reduces variance.

- **Random Forest** = bagging + DTs + per-split random subset of m features. $m \approx \sqrt{d}$ (classification) or $d/3$ (regression).
- DTs are **high-variance, low-bias**; ensembling trims the variance while keeping the flexibility.

04_Unsupervised_KMeans_PCA_Explainer

Post-Mid-Sem 4: Unsupervised Learning — K-Means + PCA

Lectures 22 (K-Means), 23 (K-Means + linear algebra review), 24 (PCA). Both are **unsupervised** — no labels y , just $\{x^{(i)}\}$. K-means discovers *clusters*; PCA discovers *directions of variation*.

Supervised vs unsupervised — the setup change

	Supervised	Unsupervised
Data	$\{(x^{(i)}, y^{(i)})\}$	$\{x^{(i)}\}$ only
Goal	Learn $f : x \mapsto y$	Discover structure in x 's
Truth label	Yes	No ground truth — evaluation is subjective
Examples	Regression, classification	Clustering (K-means), dimensionality reduction (PCA)

The “no ground truth” point matters: there’s no single right answer to “how many clusters?” or “is this a good projection?” — those are modelling choices.

Part A — K-Means clustering (L22, L23)

Problem setup

Given: - Dataset $\{x_1, \dots, x_N\}$ with $x_i \in \mathbb{R}^d$. - An integer K (chosen by the user) = desired number of clusters.

Output: a **partition** $C : \{1, \dots, N\} \rightarrow \{1, \dots, K\}$ assigning each point to exactly one of K non-overlapping groups.

The **centroid** of cluster ℓ (with m_ℓ points) is the sample mean of its points:

$$\mu_\ell = \frac{1}{m_\ell} \sum_{i \in C_\ell} x_i.$$

Objective — minimise within-cluster spread

Two equivalent ways to measure how tight a cluster is. For cluster C_ℓ with m_ℓ points:

Sum-of-pairwise-distances form (lecture’s first definition):

$$W(C_\ell) = \frac{1}{m_\ell} \sum_{i,j \in C_\ell} \|x_i - x_j\|^2.$$

Centroid form (computationally cheaper):

$$W(C_\ell) = 2 \sum_{i \in C_\ell} \|x_i - \mu_\ell\|^2.$$

These are equal up to constants — the lecture's theorem:

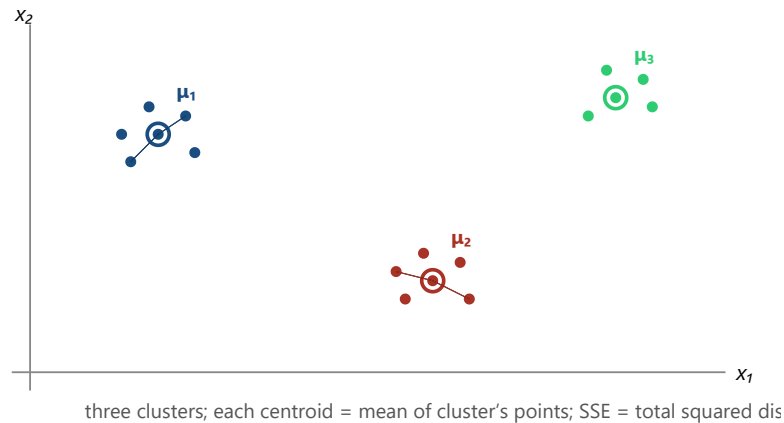
$$\sum_{i,j \in C_\ell} \|x_i - x_j\|^2 = 2 m_\ell \sum_{i \in C_\ell} \|x_i - \mu_\ell\|^2.$$

Cost comparison. Sum-of-pairs is $O(N^2)$ to compute (every pair); centroid form is $O(N)$ to find the centroid + $O(N)$ for the sum of squared distances. Use the centroid form.

Total clustering cost (the **K-means objective**, also called SSE):

$$\text{SSE}(C, \mu) = \sum_{i=1}^N \|x_i - \mu_{C(i)}\|^2.$$

Goal: choose C and $\mu_1, \dots, \mu_K$ to minimise SSE.



The K-means algorithm (Lloyd's algorithm)

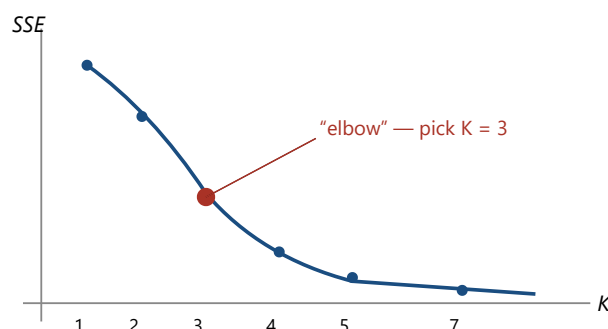
The joint minimisation over C and μ is NP-hard, so we use **alternating minimisation**:

1. Initialize K centroids $\mu_1, \dots, \mu_K$ (random points, or K random samples).
2. Repeat until convergence:
 - a) Assignment step: assign each point to its closest centroid:
 $C(i) = \operatorname{argmin}_\ell \|x_i - \mu_\ell\|^2$
 - b) Update step: recompute each centroid as the mean of its assigned points:
 $\mu_\ell = (1/m_\ell) \sum_{\{i: C(i)=\ell\}} x_i$
3. Return C, μ when assignments stop changing.

Key properties: - **Monotonic decrease:** each step never increases SSE — assignment step lowers SSE for fixed μ , update step lowers SSE for fixed C (because mean minimises sum of squared distances). So SSE decreases each iteration. - **Always converges** (in finite steps — there are only finitely many partitions of N points into K groups). - **Only to a local minimum** — outcome depends on initialisation. Standard fix: run K-means several times with different random inits, keep the run with lowest SSE.

Choosing K — the “elbow method”

There’s no statistical “right K ” — it’s a modelling choice. Heuristic: plot SSE vs K . SSE always decreases as K grows ($K = N$ gives $SSE = 0$). The plot usually has an **elbow** — a value of K past which adding more clusters only marginally reduces SSE. Pick the elbow.



Part B — Principal Component Analysis (L24)

Problem setup

n data points $x_1, \dots, x_n \in \mathbb{R}^d$. We want to **reduce dimension** from d to $p \ll d$ — replace each $x^{(i)}$ with a shorter vector $\tilde{x}^{(i)} \in \mathbb{R}^p$ that preserves as much *information* as possible.

Reasons to do this: - **Computation / storage**: smaller vectors are faster and cheaper. - **Visualisation**: project to $p = 2$ or $p = 3$ to look at the data. - **Redundancy**: many features are correlated; PCA finds the underlying lower-dimensional structure.

The PCA criterion: choose the lower-dimensional representation that **maximises the variance** of the projected data. Why variance? A direction in which the data spreads out tells us the most about it; a direction in which all points look the same is uninformative.

Wrong: project to axis x_1 projected pts: large spread along x_1 ? Yes, but wastes info

Right: project to principal direction projected pts spread maximally — captures all the variation

The optimisation problem

Suppose we project each centered x_i onto a unit vector $u_1 \in \mathbb{R}^d$. The projection (scalar) is $u_1^\top x_i$. The empirical variance of the projected points is

$$\text{Var}(u_1^\top x) = u_1^\top S u_1,$$

where $S = \frac{1}{n} \sum_i (x_i - \bar{x})(x_i - \bar{x})^\top$ is the **sample covariance matrix** of the data.

PCA optimisation (find first principal direction):

$$\max_{u_1} u_1^\top S u_1 \quad \text{s.t.} \quad \|u_1\| = 1.$$

The constraint $\|u_1\| = 1$ is necessary because otherwise scaling u_1 up indefinitely makes the objective unbounded.

Solution — eigenvector of largest eigenvalue

Spectral theorem fact. For a symmetric matrix S with eigenvalue decomposition $S = U\Lambda U^\top$ (eigenvalues $\lambda_1 \geq \lambda_2 \geq \dots$ with orthonormal eigenvectors $u_1, u_2, \dots$):

$$\max_{\|u\|=1} u^\top S u = \lambda_1, \quad \text{achieved at } u = u_1.$$

So **the first principal direction u_1 is the eigenvector of S corresponding to the largest eigenvalue λ_1 .**

For p principal directions: $u_1, \dots, u_p$ are the top- p eigenvectors of S , sorted by eigenvalue descending. They are pairwise orthogonal (because S is symmetric $\Rightarrow$ eigenvectors are orthogonal).

The PCA algorithm

1. Center the data: $x_i \leftarrow x_i - \bar{x}$ for all i .
2. Compute sample covariance $S = (1/n) X^\top X$ (where rows of X are the centered x_i).
3. Compute eigendecomposition of $S \rightarrow$ eigenvalues $\lambda_1 \geq \dots \geq \lambda_d$, eigenvectors $u_1, \dots, u_d$.
4. Pick top p eigenvectors: $U_p = [u_1 \mid \dots \mid u_p]$.
5. Project each x_i to p -dim: $\tilde{x}_i = U_p^\top x_i \in \mathbb{R}^p$.

Computing eigenvalues by hand (for $d = 2$)

For $A = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$, the **characteristic equation** is

$$\det(A - \lambda I) = (a - \lambda)(d - \lambda) - bc = 0.$$

Solve the quadratic for λ . Plug each λ back into $(A - \lambda I)v = 0$ to get v , then normalise.

For an $m \times m$ matrix, the characteristic polynomial has degree m , so at most m (real) roots.

How much variance does PCA preserve?

The total variance in the data is $\text{trace}(S) = \sum_i \lambda_i$. After keeping the top p components, the **explained variance ratio** is

$$\frac{\sum_{i=1}^p \lambda_i}{\sum_{i=1}^d \lambda_i}.$$

Common choice: pick smallest p such that this ratio ≥ 0.95 .

Part C — Worked sample problems

Problem 1 — One full iteration of K-means

Points: $A = (1, 1)$, $B = (1, 2)$, $C = (5, 5)$, $D = (5, 6)$. Initialise centroids $\mu_1 = (1, 1)$, $\mu_2 = (2, 2)$. Run one full iteration of K-means.

Solution. *Assignment:* distances to each centroid (squared): - $A = (1, 1)$: d^2 to $\mu_1 = 0$, to $\mu_2 = 2$. Assign to cluster 1. - $B = (1, 2)$: d^2 to $\mu_1 = 1$, to $\mu_2 = 1$. Tie — say cluster 1. - $C = (5, 5)$: d^2 to $\mu_1 = 32$, to $\mu_2 = 18$. Cluster 2. - $D = (5, 6)$: d^2 to $\mu_1 = 41$, to $\mu_2 = 25$. Cluster 2.

Update: $\mu_1 = ((1 + 1)/2, (1 + 2)/2) = (1, 1.5)$. $\mu_2 = ((5 + 5)/2, (5 + 6)/2) = (5, 5.5)$. Already converged structurally.

Problem 2 — SSE before and after

For the converged state above, compute total SSE.

Solution. - Cluster 1: $\|A - \mu_1\|^2 = 0 + 0.25 = 0.25$; $\|B - \mu_1\|^2 = 0 + 0.25 = 0.25$. Sum 0.5. - Cluster 2: $\|C - \mu_2\|^2 = 0 + 0.25 = 0.25$; $\|D - \mu_2\|^2 = 0 + 0.25 = 0.25$. Sum 0.5. - Total SSE = 1.0.

Problem 3 — Why K-means cannot find non-convex clusters

K-means is biased toward roughly spherical clusters of similar size. Why?

Solution. The assignment step partitions space by **Voronoi cells** of the centroids — convex regions whose boundaries are hyperplanes. So K-means can only carve space into convex chunks. Two crescents arranged like a yin-yang have non-convex true clusters; K-means cannot recover them no matter how good the initialisation. (Solution: use density-based methods like DBSCAN, or kernel K-means.)

Problem 4 — Compute eigenvalues of a 2×2 matrix

Find eigenvalues of $A = \begin{pmatrix} 4 & 1 \\ 2 & 3 \end{pmatrix}$.

Solution. $\det(A - \lambda I) = (4 - \lambda)(3 - \lambda) - 2 = \lambda^2 - 7\lambda + 10 = 0$. Roots:
 $\lambda = (7 \pm \sqrt{49 - 40})/2 = (7 \pm 3)/2 = 5, 2$. So $\lambda_1 = 5$, $\lambda_2 = 2$.

Eigenvector for $\lambda = 5$: $(A - 5I)v = 0 \Rightarrow \begin{pmatrix} -1 & 1 \\ 2 & -2 \end{pmatrix} v = 0$, so $v_1 = v_2$, take $v = (1, 1)$, normalise to $(\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}})$.

Problem 5 — PCA on a 2D dataset

Three points: $(1, 1)$, $(2, 2)$, $(3, 3)$. Find the first principal direction.

Solution. Center: mean $(2, 2)$. Centered points: $(-1, -1)$, $(0, 0)$, $(1, 1)$.

$$S = \frac{1}{3} \sum (x_i - \bar{x})(x_i - \bar{x})^\top = \frac{1}{3} \left[\begin{pmatrix} -1 \\ -1 \end{pmatrix} (-1, -1) + 0 + \begin{pmatrix} 1 \\ 1 \end{pmatrix} (1, 1) \right] = \frac{1}{3} \cdot \begin{pmatrix} 2 & 2 \\ 2 & 2 \end{pmatrix} = \begin{pmatrix} 2/3 & 2/3 \\ 2/3 & 2/3 \end{pmatrix}.$$

Eigenvalues: $\det(S - \lambda I) = (2/3 - \lambda)^2 - (2/3)^2 = \lambda^2 - (4/3)\lambda = 0 \Rightarrow \lambda = 0, 4/3$. First principal eigenvector ($\lambda = 4/3$): $(S - (4/3)I)v = 0$ gives $-(2/3)v_1 + (2/3)v_2 = 0 \Rightarrow v_1 = v_2$. Normalised: $u_1 = (\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}})$.

That's the 45° line — the data sits on this line, so all variance is along it. Explained variance $= 4/3 / (4/3) = 100\%$.

Problem 6 — Explained variance

Eigenvalues of S are $\lambda = 10, 5, 3, 1, 0.5$. How many components to keep for 95% explained variance?

Solution. Total = 19.5. Need $\geq 0.95 \cdot 19.5 = 18.525$. - $p = 1$: $10/19.5 = 0.513$. - $p = 2$: $15/19.5 = 0.769$. - $p = 3$: $18/19.5 = 0.923$. - $p = 4$: $19/19.5 = 0.974$. ✓

Keep $p = 4$ components.

Problem 7 — K-means cost-of-step argument

Prove that the K-means update step (recomputing centroids) does not increase SSE.

Solution. For fixed assignment C , SSE separates per cluster:

$$\text{SSE} = \sum_{\ell} \sum_{i \in C_{\ell}} \|x_i - \mu_{\ell}\|^2.$$

Each term $\sum_{i \in C_{\ell}} \|x_i - \mu_{\ell}\|^2$ is minimised over μ_{ℓ} at $\mu_{\ell} = (1/m_{\ell}) \sum_{i \in C_{\ell}} x_i$ (the mean) — standard 1-D and multivariate fact (derivative of sum of squares set to zero). So the update step replaces each μ_{ℓ} with its argmin, weakly decreasing SSE. The assignment step also weakly decreases SSE (each point gets assigned to its closest centroid). Both steps monotonic $\Rightarrow$ algorithm converges. (To a *local* minimum, not necessarily global.)

Problem 8 — Why centre the data before PCA?

What goes wrong if you run PCA without first centring?

Solution. Without centring, the formula $S = (1/n)X^{\top}X$ is no longer the covariance — it's the "second moment" matrix. Its top eigenvector points roughly to the **direction of the mean** rather than the direction of maximum variation around the mean. For data clustered far from the origin, you'd be capturing "where the cloud sits" instead of "how the cloud spreads." Centring removes the mean offset so the first principal direction captures pure variation.

Part D — Cheat-sheet take-aways

- **Unsupervised:** no labels, no ground truth.
- **K-means objective:** $\text{SSE}(C, \mu) = \sum_i \|x_i - \mu_{C(i)}\|^2$. Centroid form is $O(N)$, pairwise form is $O(N^2)$ — they're equivalent up to a constant.
- **K-means algorithm:** alternate (1) assignment $C(i) = \arg \min_{\ell} \|x_i - \mu_{\ell}\|^2$ and (2) update $\mu_{\ell} = \text{mean of cluster } \ell$. Monotone decrease in SSE; converges to a local minimum; multiple restarts recommended.
- **K-means bias:** Voronoi cells $\Rightarrow$ only convex clusters of similar size.
- **PCA objective:** $\max_{\|u\|=1} u^{\top} S u$ where $S = \text{sample covariance}$.
- **PCA solution:** top- p eigenvectors of S (sorted by eigenvalue descending). $u_1 \leftrightarrow$ largest λ_1 .
- **Workflow:** (1) centre data, (2) compute S , (3) eigen-decompose, (4) project: $\tilde{x}_i = U_p^{\top} x_i$.
- **Explained variance ratio:** $\sum_{i=1}^p \lambda_i / \sum_{i=1}^d \lambda_i$.
- **Eigenvalue recipe:** solve $\det(A - \lambda I) = 0$, then $(A - \lambda I)v = 0$ for each λ .